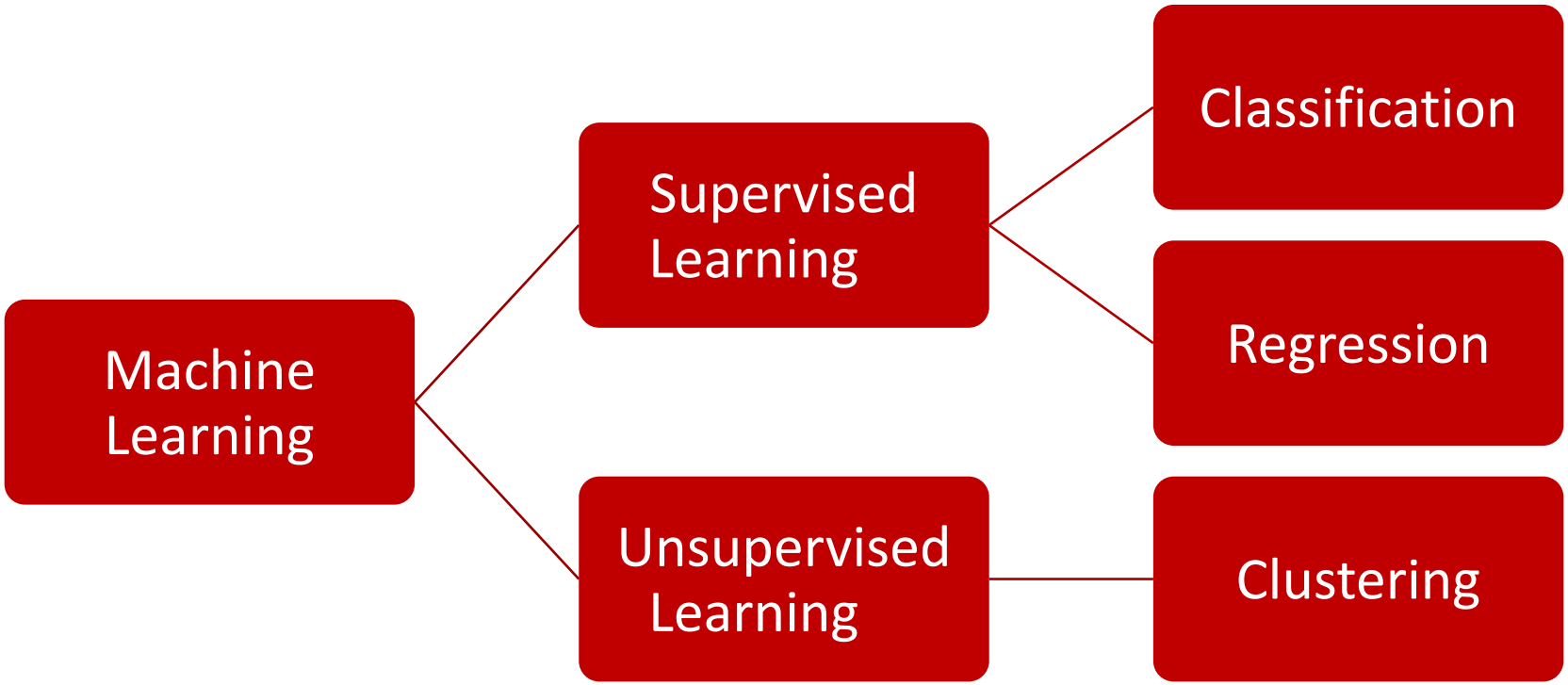
L05 - L07 – KNN

Data Mining
Dr Andrea Visentin

Thanks to Dr Alejandro Arbelaez
And to Flaticon.com

Machine learning

Algorithm Hierarchy



Supervised learning

In supervised learning the dataset the dataset contains labels / desired output.

Data

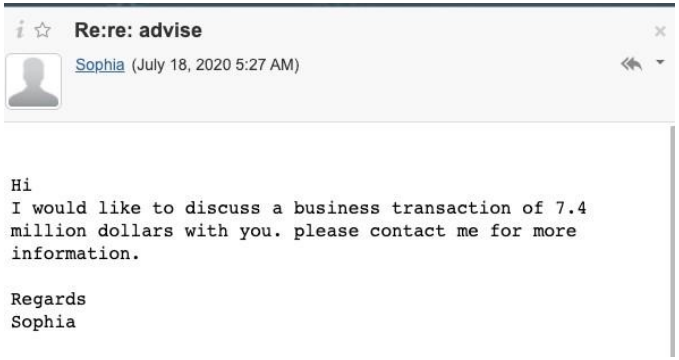
Label

Supervised learning

Problem	Attributes	Label
Credit Card Fraud Detection	Transaction detail, user detail	Normal Transaction/Fraud
Spam filter	Email text, sender	SPAM/ No-SPAM
Face recognition	Photo of the face	Name of the individual
Tumor detection	MRI, blood tests	Presence and location
Stock market price prediction	Company financial statement, price history	Tomorrow's price
Sentiment analysis	Text of a review/comment	Sentiments associated with that review/comment

SPAM filter

Supervised learning is the **machine learning** task of **learning** a function that maps an input to an output based on example input-output pairs



Re:re: advise

Sophia (July 18, 2020 5:27 AM)

Hi
I would like to discuss a business transaction of 7.4 million dollars with you. please contact me for more information.

Regards
Sophia

Spam

UCC all-staff survey on experience working during the covid-19 pandemic

President
Wed 27/01/2021 14:45
To: All Staff

Dear Colleagues,

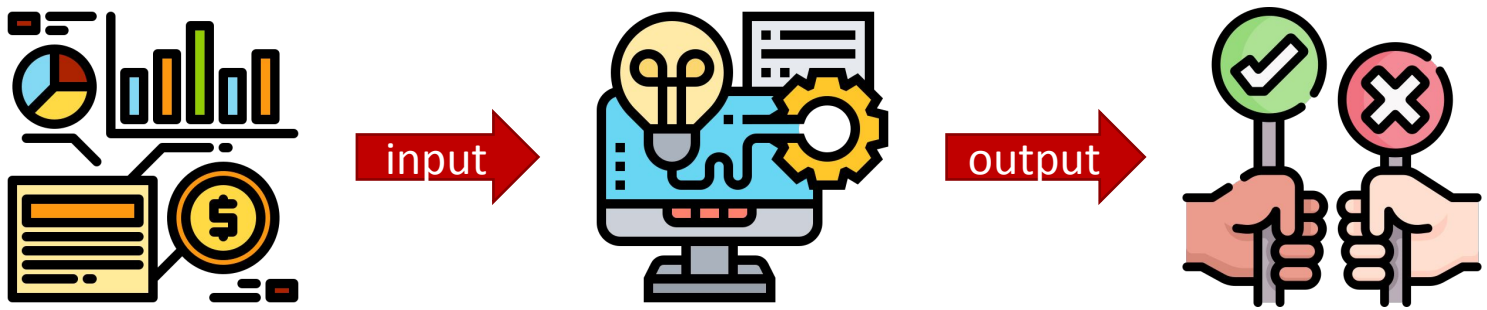
The beginning of term this week marks our third semester of online teaching and remote working. While new vaccines bring much-needed hope for 2021, our resilience will continue to be tested in the coming months. Many of us feel fatigue as we remain isolated from each other, while those of us with caring obligations may be under heightened stress. Remote working has brought unanticipated benefits for some, while others may be struggling, and concerned about their careers.

As we embark on UCC2022, our strategic pivot for the next two years, University managers and leaders want to better understand your experience of remote working, to ensure the policies and supports we develop in response to the pandemic, recognise and meet your needs.

Not Spam

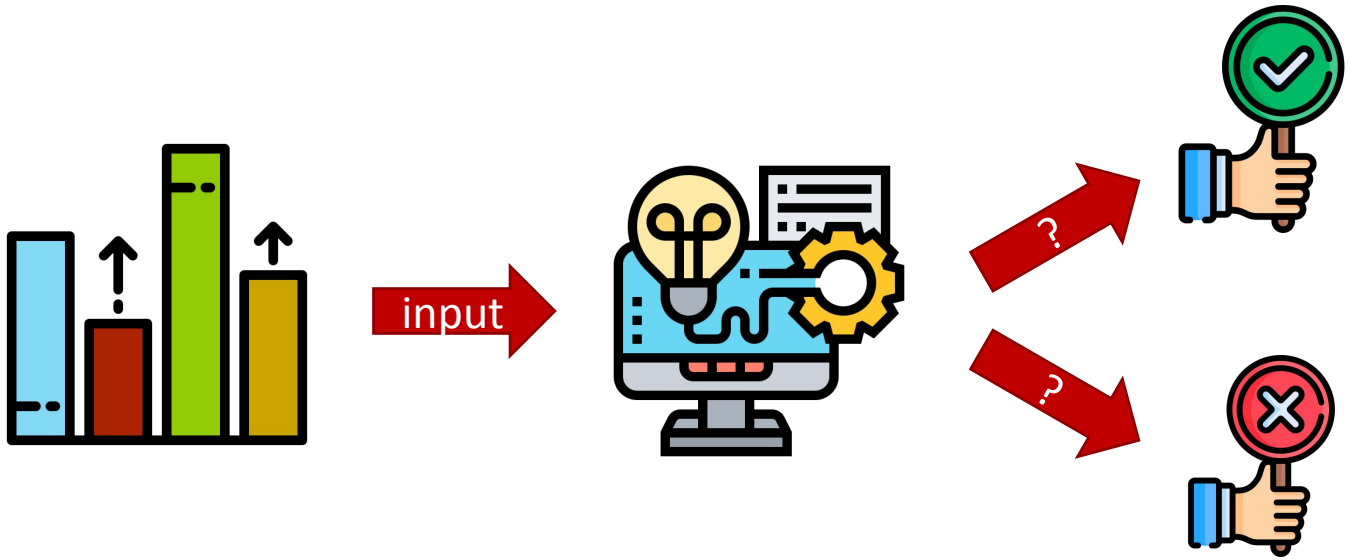
Supervised learning

The machine learning model learns how to infer the desired output based on the input.



Supervised learning

The model then receives previously unseen data and try to infer the label with as much accuracy as possible



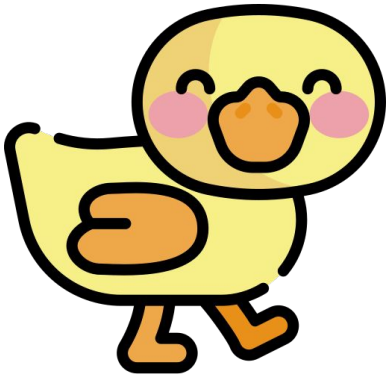
Classification

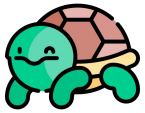
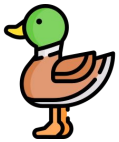
- Given a collection of records (**training set**)
  - Each record contains a set of attributes, one of the attributes is the class
- Find a **model** for class attribute as a function of the values of other attributes
- Goal: **previously unseen** records should be assigned a class as accurately as possible
  - A **test set** is used to determine the accuracy of the model. Usually, the given data set is divided into training and test sets, with training set used to build the model and test set used to validate it

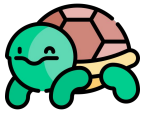
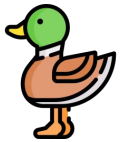
Nearest Neighbour Classification

Nearest Neighbour Classifier

- Basic idea:
  - https://en.wikipedia.org/wiki/Duck_test
  - If it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck.



Attribute	Label
	Fox
	Panda
	Flamingo
	Duck
	Turtle
	Turtle
	Duck

Attribute		Label
	→	Fox
	→	Panda
	→	Flamingo
	→	Duck
	→	Turtle
	→	Turtle
	→	Duck

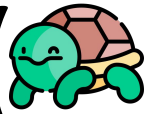
f( ) = Fox

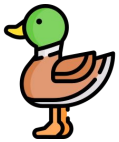
f( ) = Panda

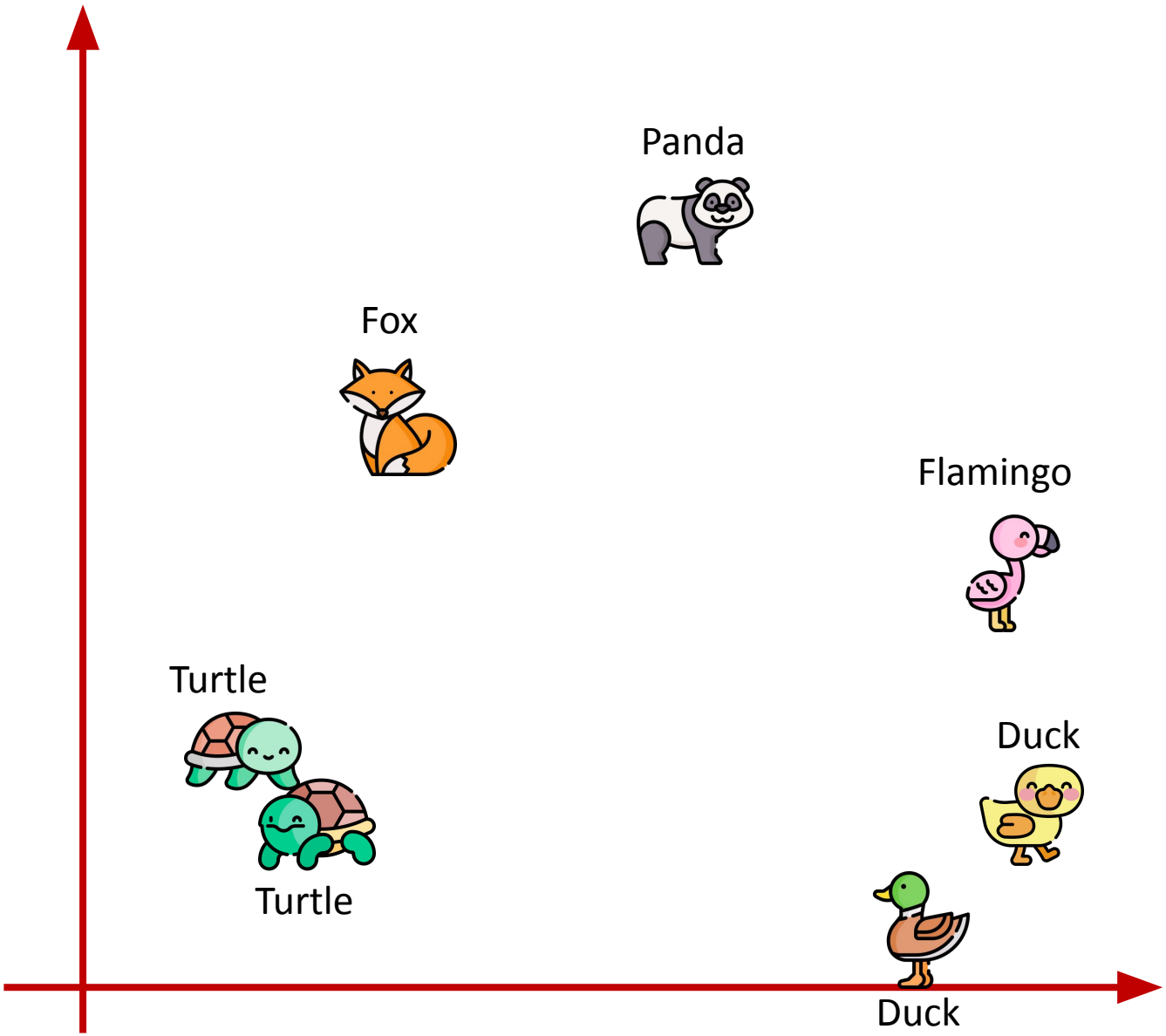
f( ) = Flamingo

f( ) = Duck

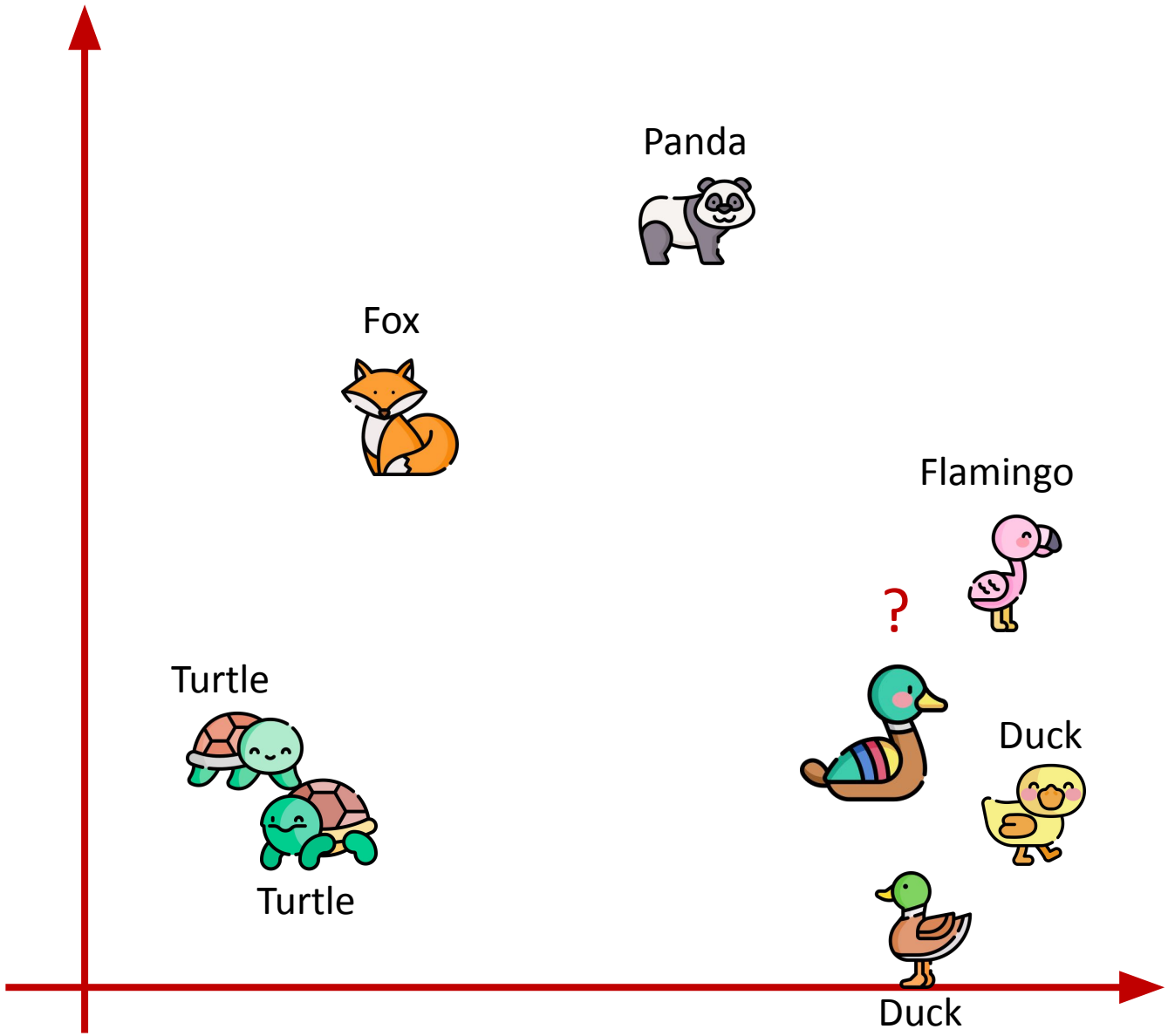
f( ) = Turtle

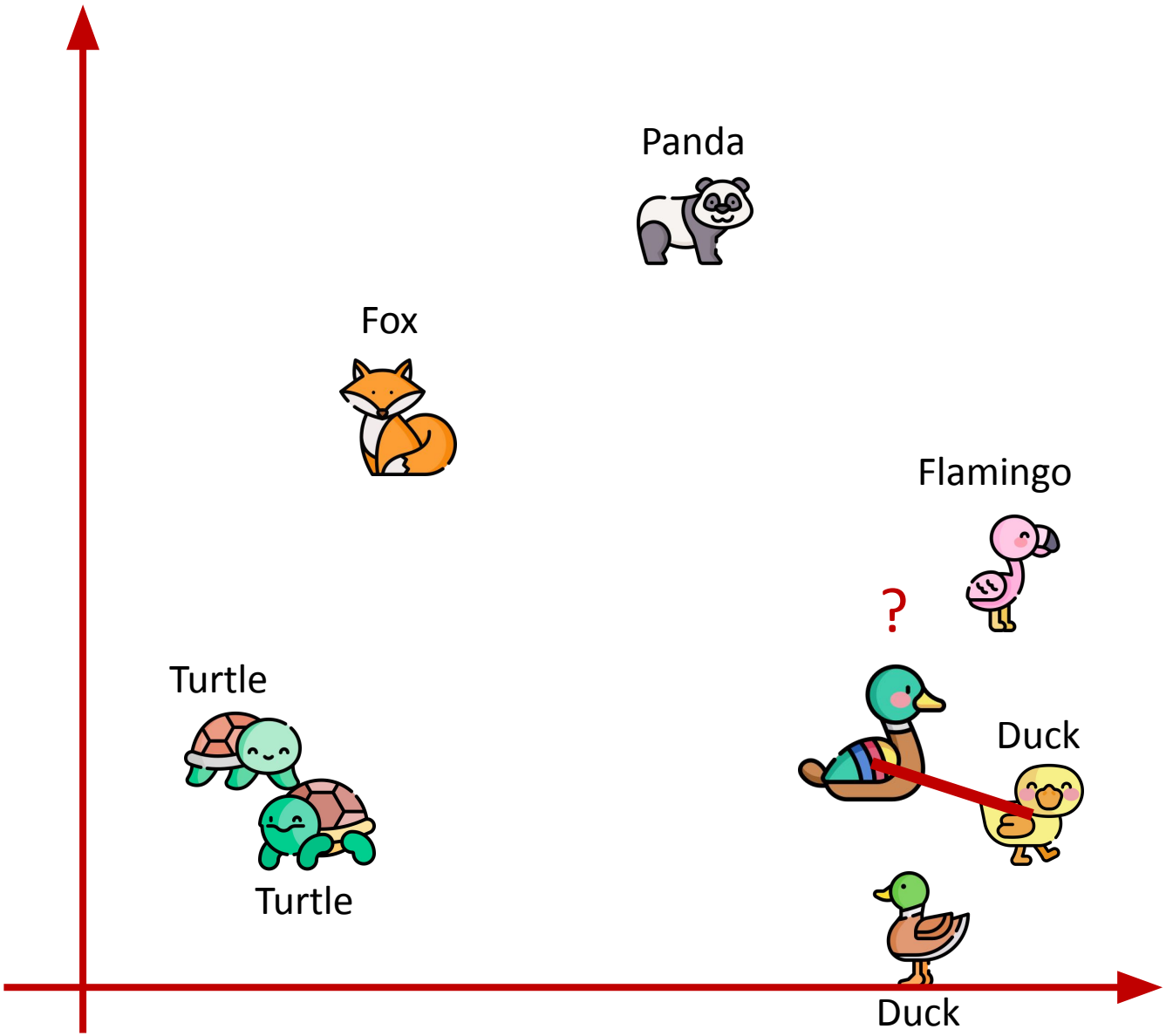
f() = Turtle

f() = Duck


Panda
Fox
Flamingo
Turtle
Duck
Turtle
Duck

f( ) =


Panda
Fox
Flamingo
?
Turtle
Duck
Turtle
Duck


Panda
Fox
Flamingo
?
Turtle
Duck
Turtle
Duck

Close to

Duck

It is

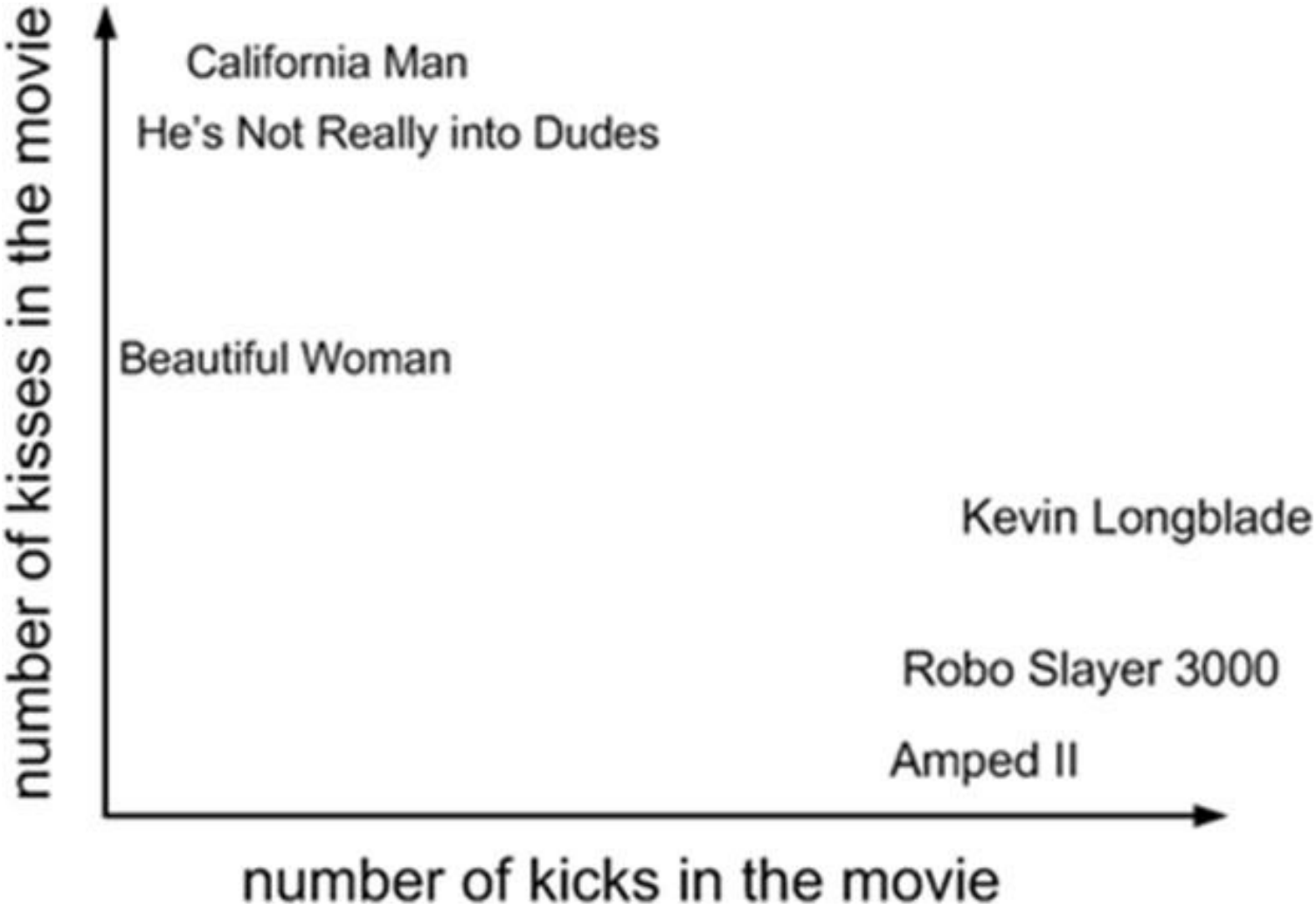
Duck!

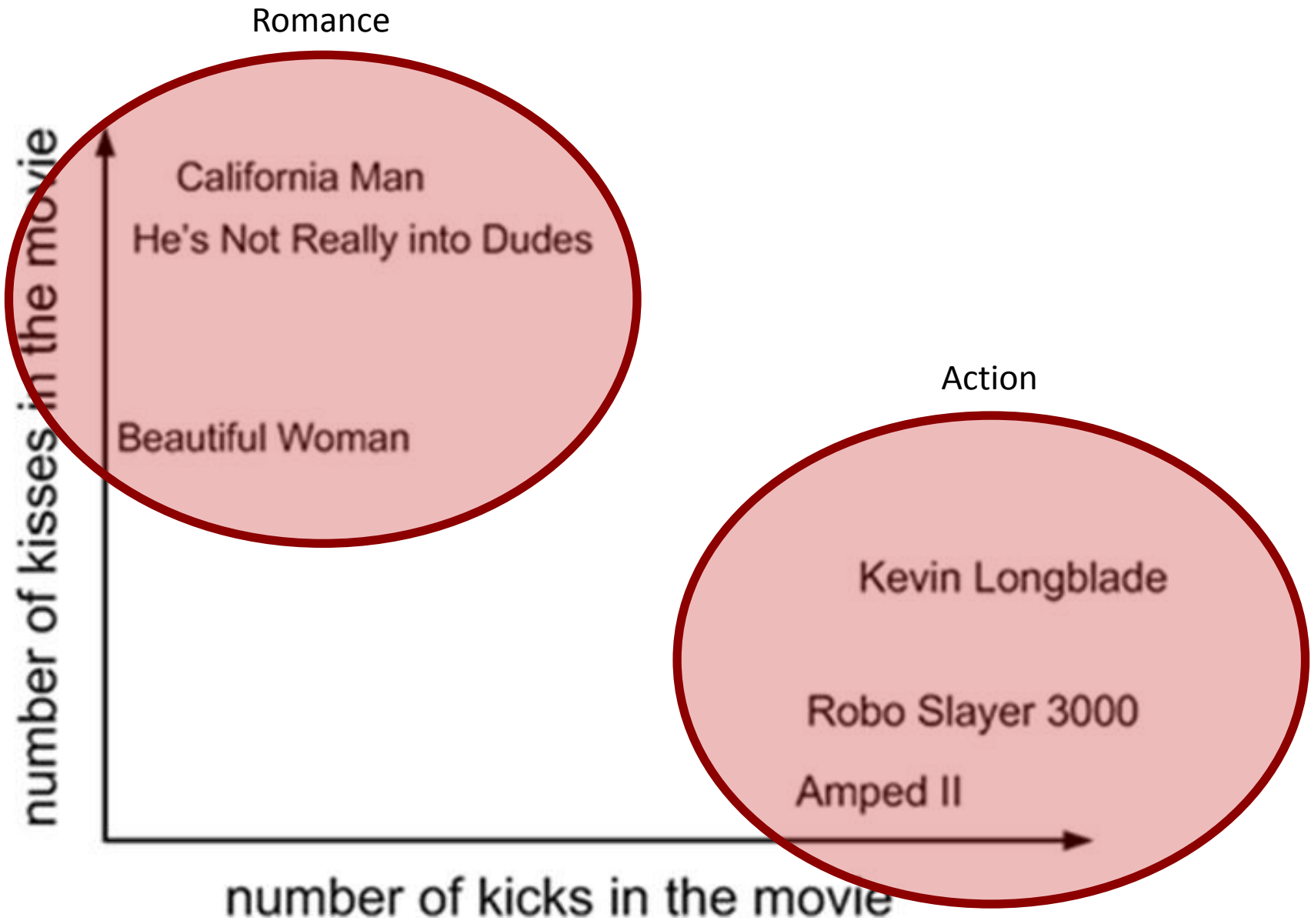
f( ) = duck

Sample Dataset

Movie title	# of kicks	# of kisses	Type of movie
California Man	3	104	Romance
He's Not Really into Dudes	2	100	Romance
Beautiful Woman	1	81	Romance
Kevin Longblade	101	10	Action
Robo Slayer 3000	99	5	Action
Amped II	98	2	Action

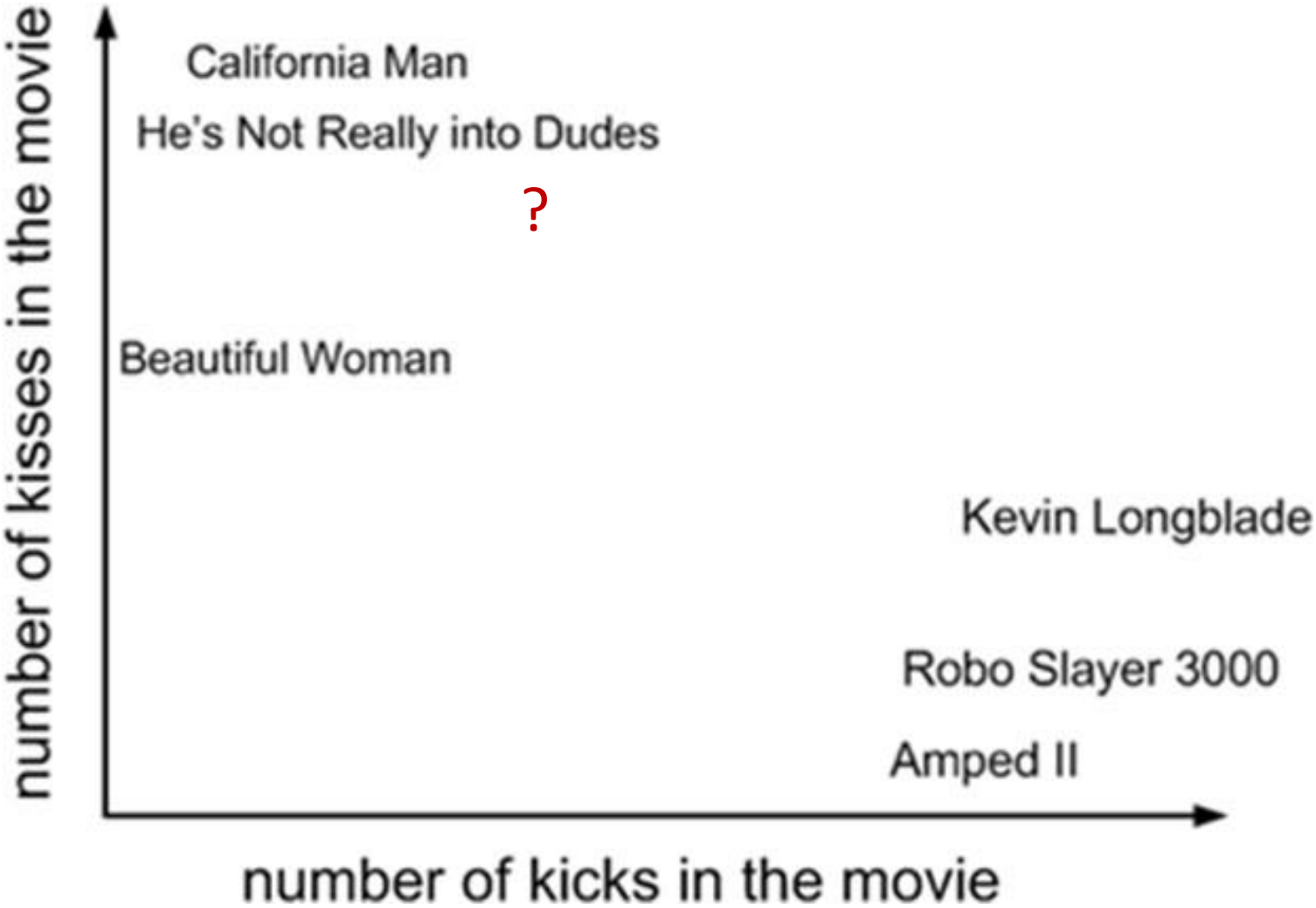
Machine Learning In Action – Peter Harrington


number of kisses in the movie
California Man
He's Not Really into Dudes
Beautiful Woman
Kevin Longblade
Robo Slayer 3000
Amped II
number of kicks in the movie


Romance
California Man
He's Not Really into Dudes
Beautiful Woman
Action
Kevin Longblade
Robo Slayer 3000
Amped II
number of kisses in the movie
number of kicks in the movie

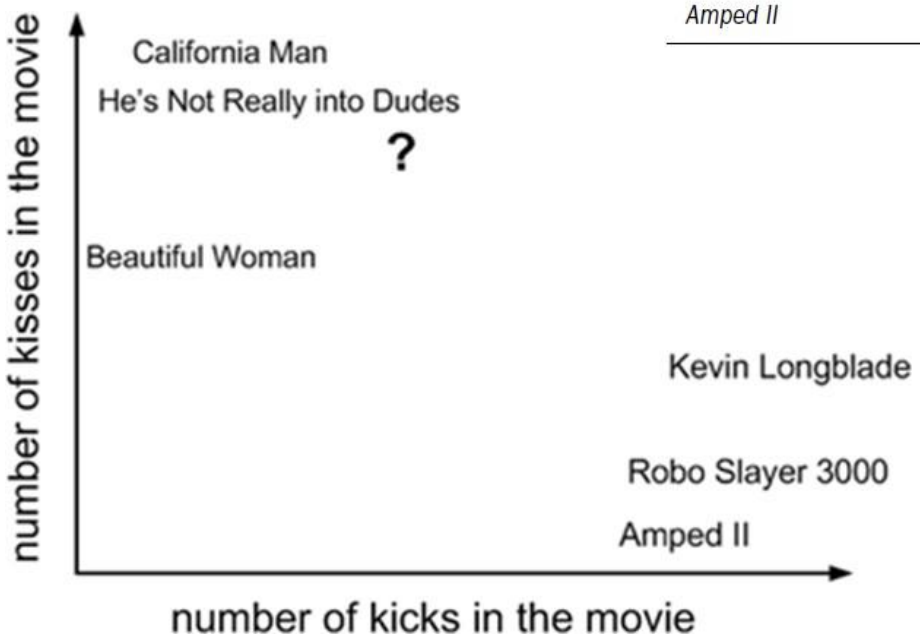
Movie title	# of kicks	# of kisses	Type of movie
California Man	3	104	Romance
He's Not Really into Dudes	2	100	Romance
Beautiful Woman	1	81	Romance
Kevin Longblade	101	10	Action
Robo Slayer 3000	99	5	Action
Amped II	98	2	Action
?	18	90	Unknown

Assume we get an unseen movie and we have to classify it as Romance or action based on it's attribute values.


number of kisses in the movie
California Man
He's Not Really into Dudes
?
Beautiful Woman
Kevin Longblade
Robo Slayer 3000
Amped II
number of kicks in the movie

Given a query instance x_q,
first locate the nearest training example x_n
then $f(x_q) := f(x_n)$

Movie title	Distance to movie "?"
California Man	20.5
He's Not Really into Dudes	18.7
Beautiful Woman	19.2
Kevin Longblade	115.3
Robo Slayer 3000	117.4
Amped II	118.9



As the query is closest to the film "He's Not Really into Dudes" then it is classified as a **Romance**

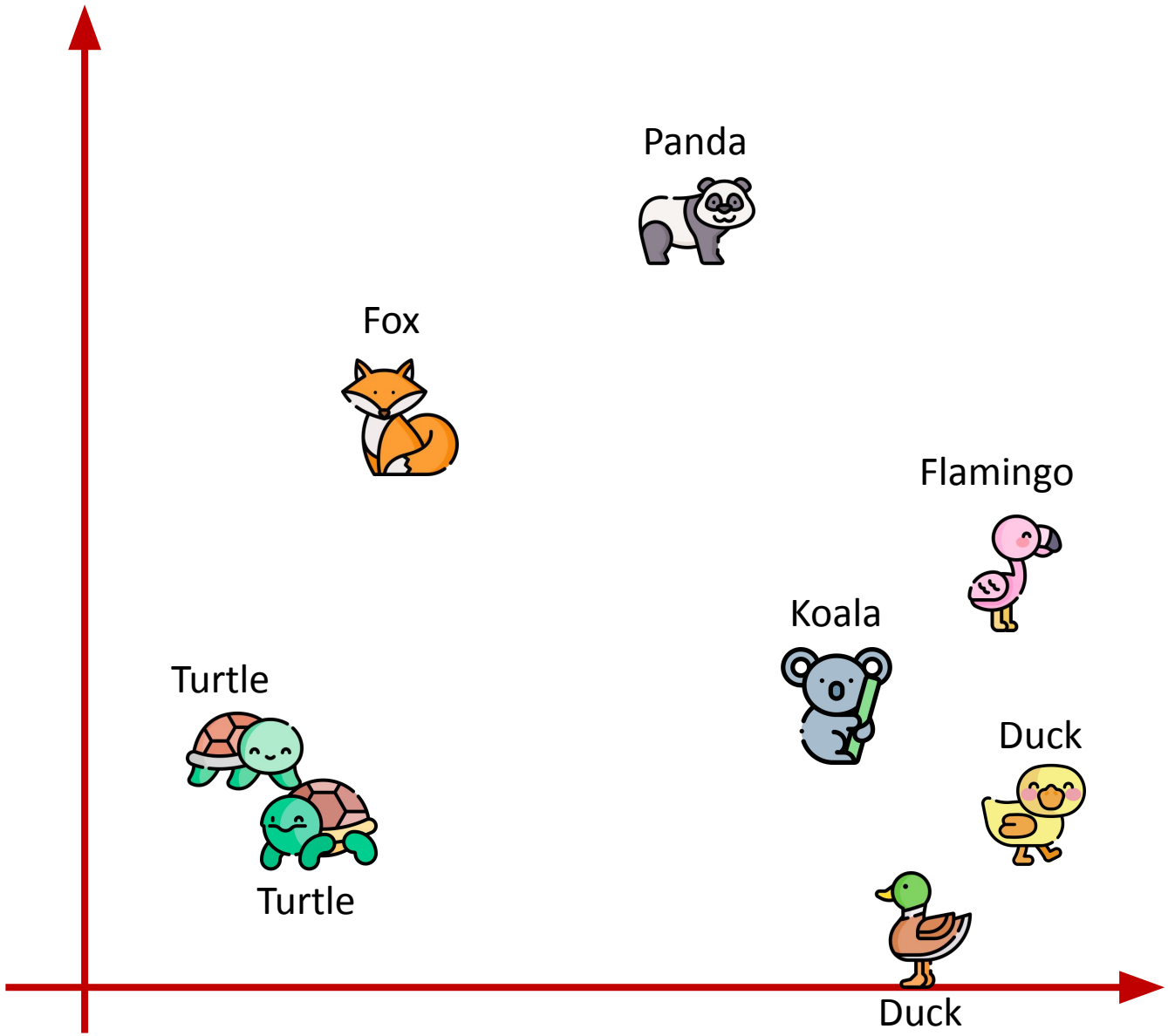
Can you see any problems with this simple approach?

Missclassification

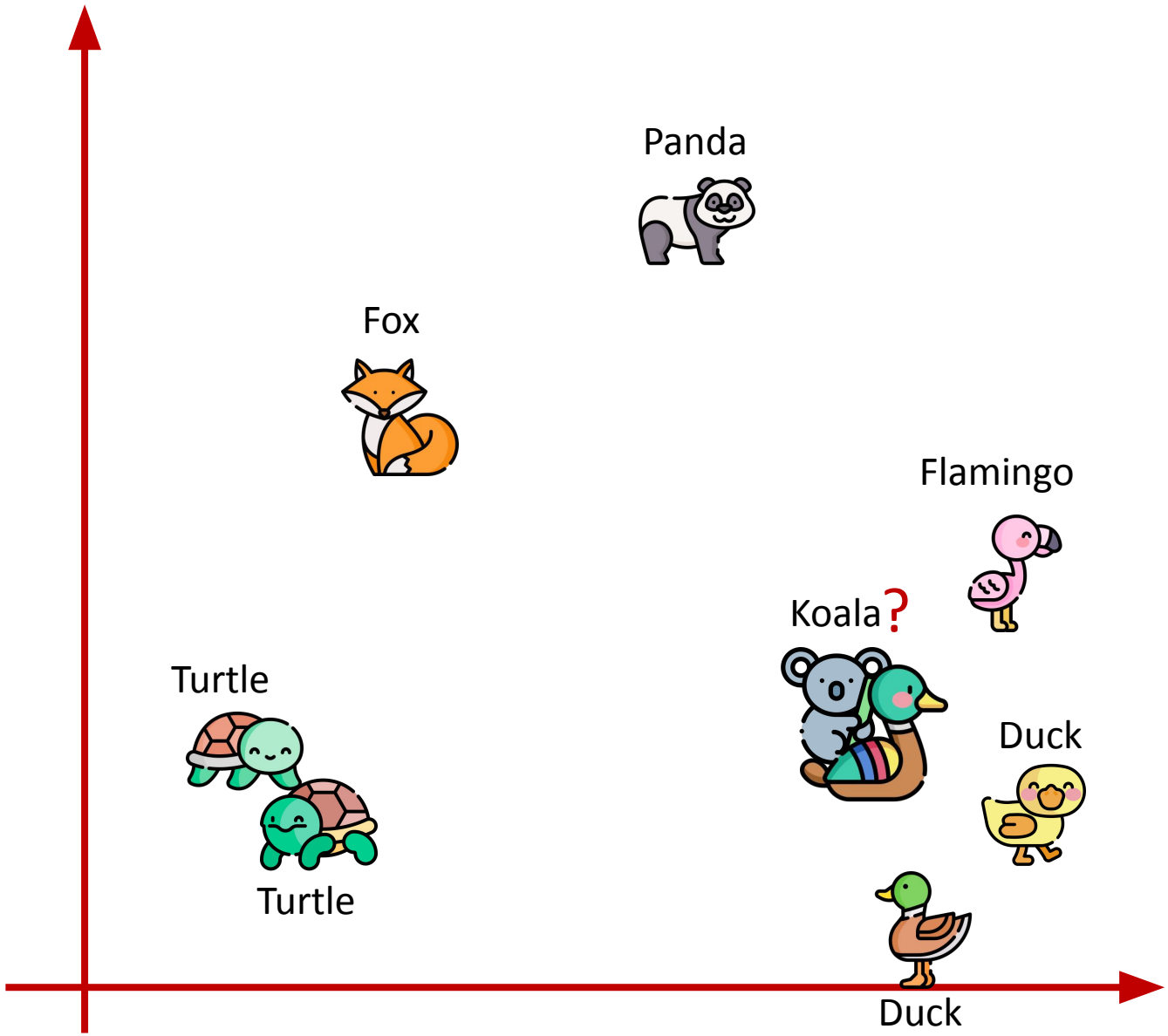
- What would happen if "He's no really into dudes" was incorrectly classified as an Action movie.
  - Our new query instance '?' would also get incorrectly classified as an Action. Any ideas of how we might go about improving the algorithm?

Movie title	# of kicks	# of kisses	Type of movie
California Man	3	104	Romance
He's Not Really into Dudes	2	100	Action
Beautiful Woman	1	81	Romance
Kevin Longblade	101	10	Action
Robo Slayer 3000	99	5	Action
Amped II	98	2	Action
?	18	90	Unknown

Noise: An incorrect classificatio n


Panda
Fox
Flamingo
Koala
Turtle
Duck
Turtle
Duck

f( ) =


Panda
Fox
Flamingo
Koala?
Turtle
Duck
Turtle
Duck

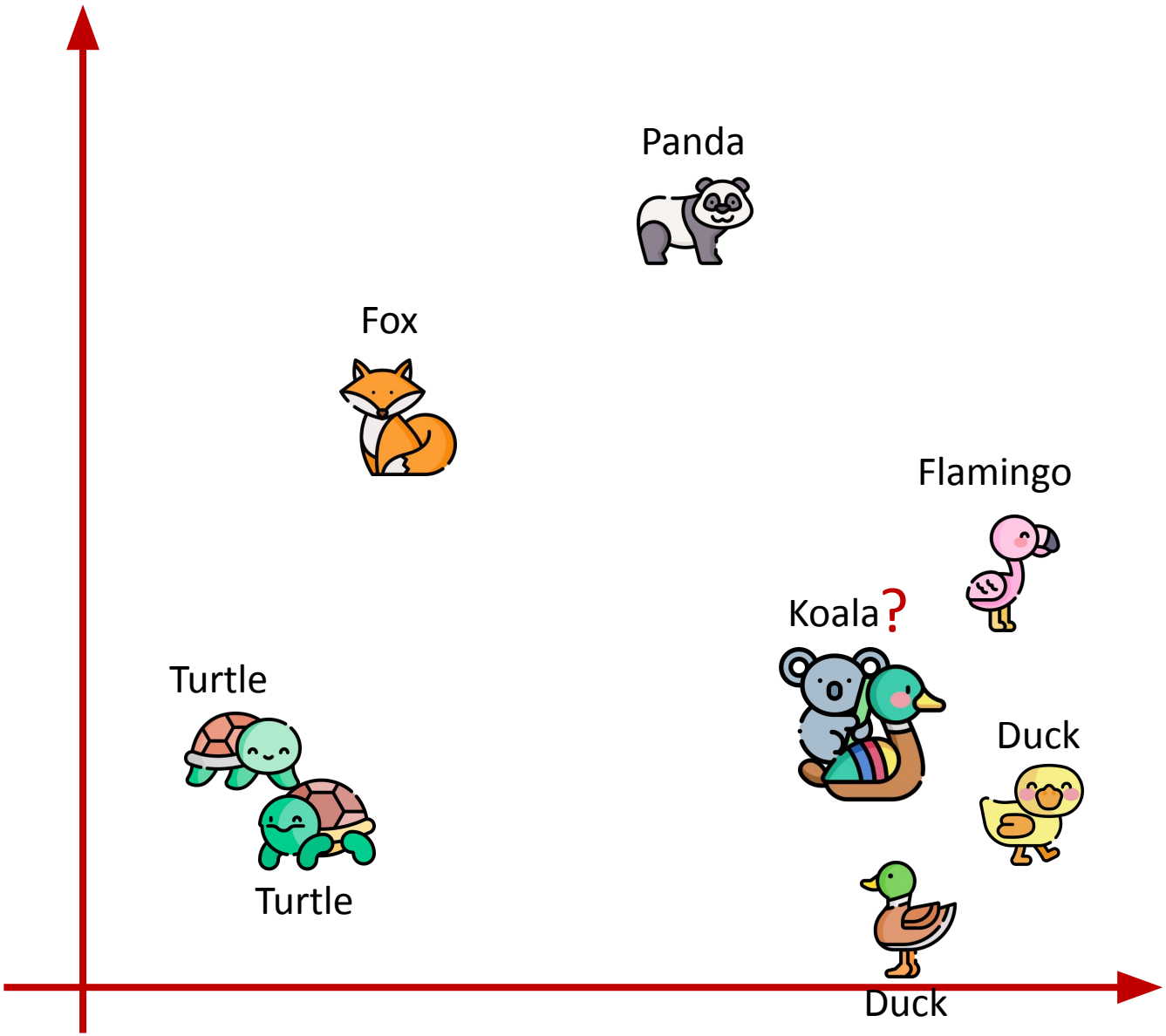
Close to

Koala

It is

Koala!

f( ) = koala


Panda
Fox
Flamingo
Koala?
Turtle
Duck
Turtle
Duck

Close to

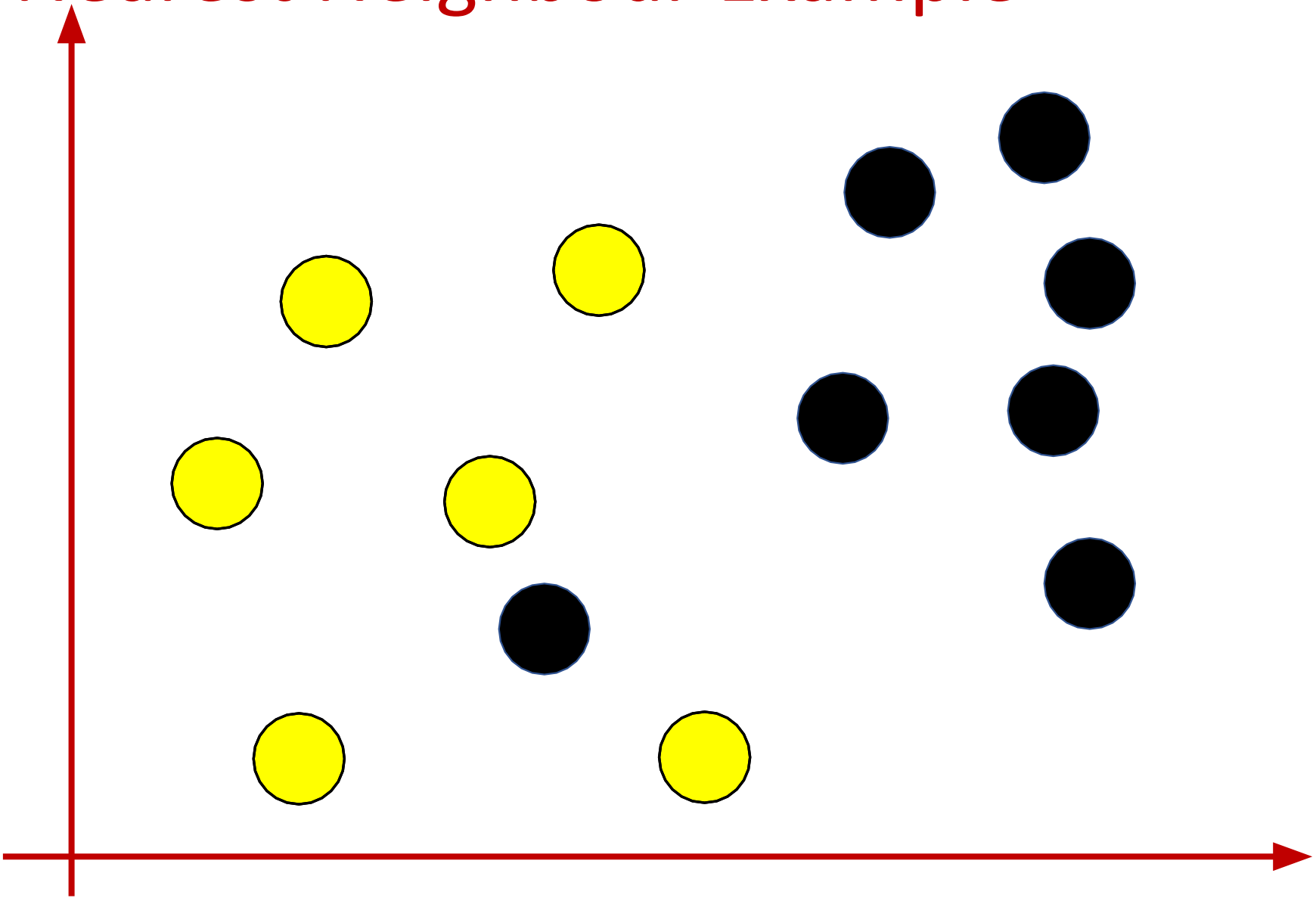
2 Duck
1 Koala

It is

Duck!

f( ) = duck

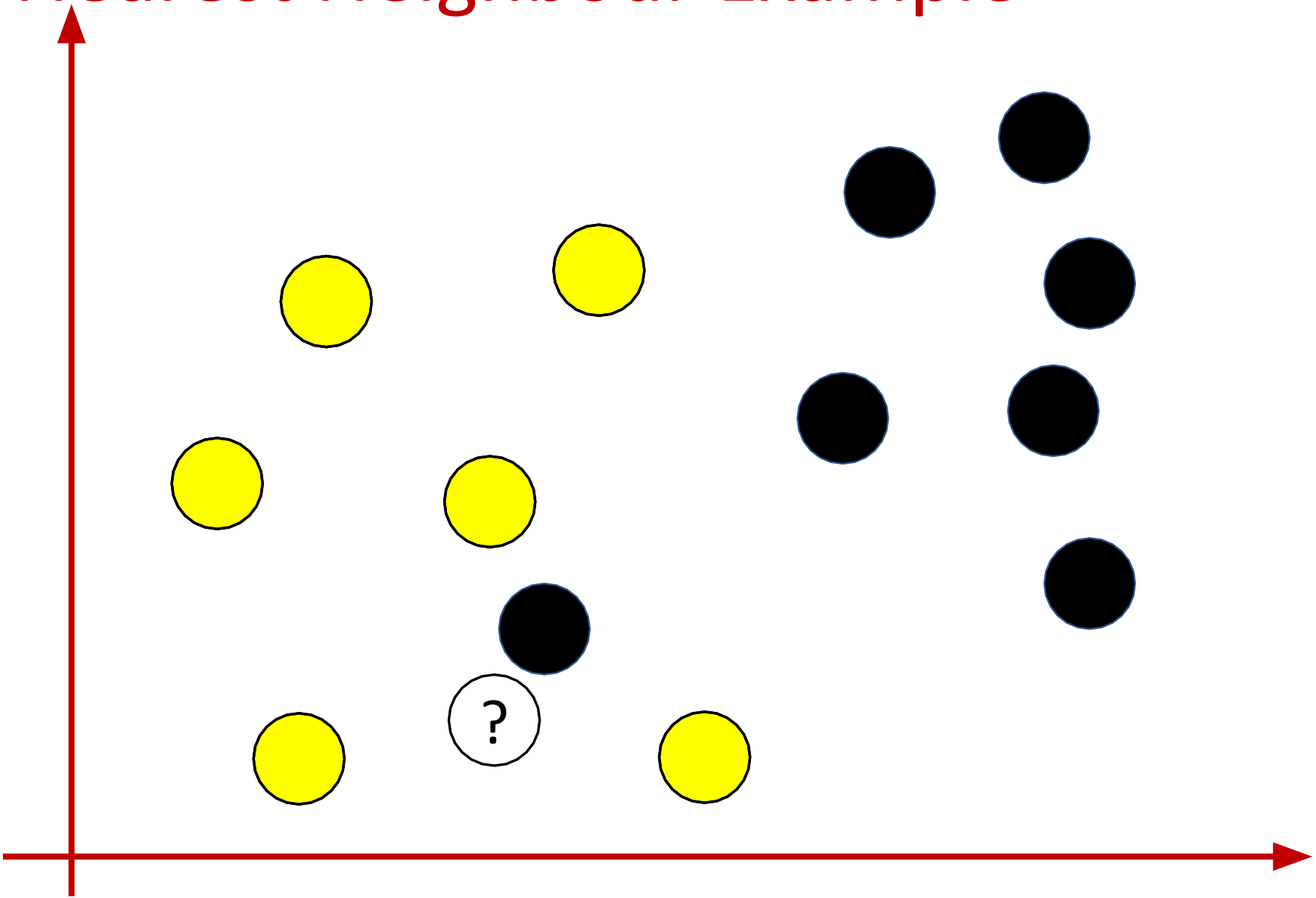
Nearest Neighbour Example



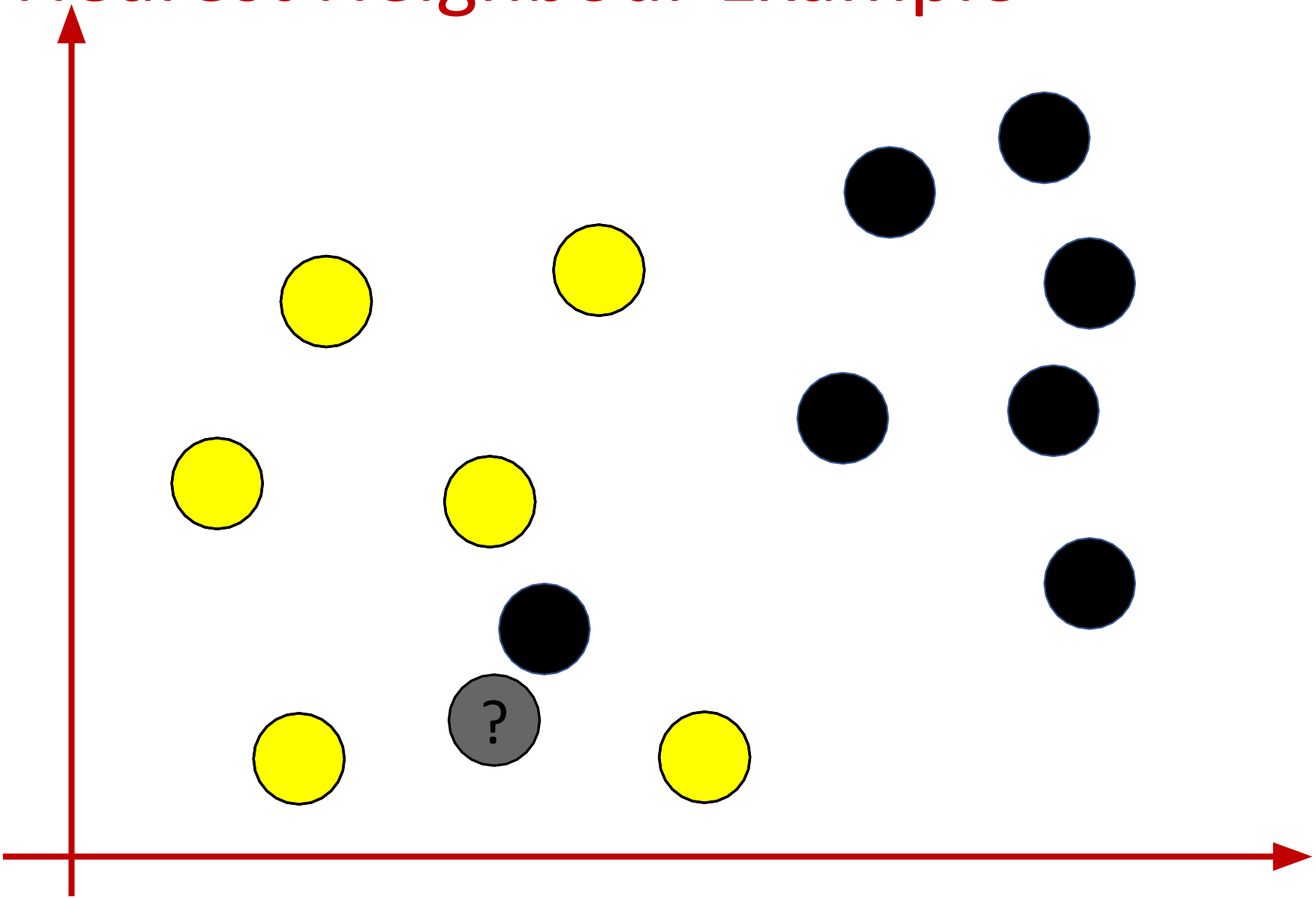
Nearest Neighbour Example

- We wish to classify the new case, which is the white circle with the question mark.
- The nearest neighbour is selected and the example instance is classified as same.
- Is the new instance classified as a black or yellow circle?

Nearest Neighbour Example



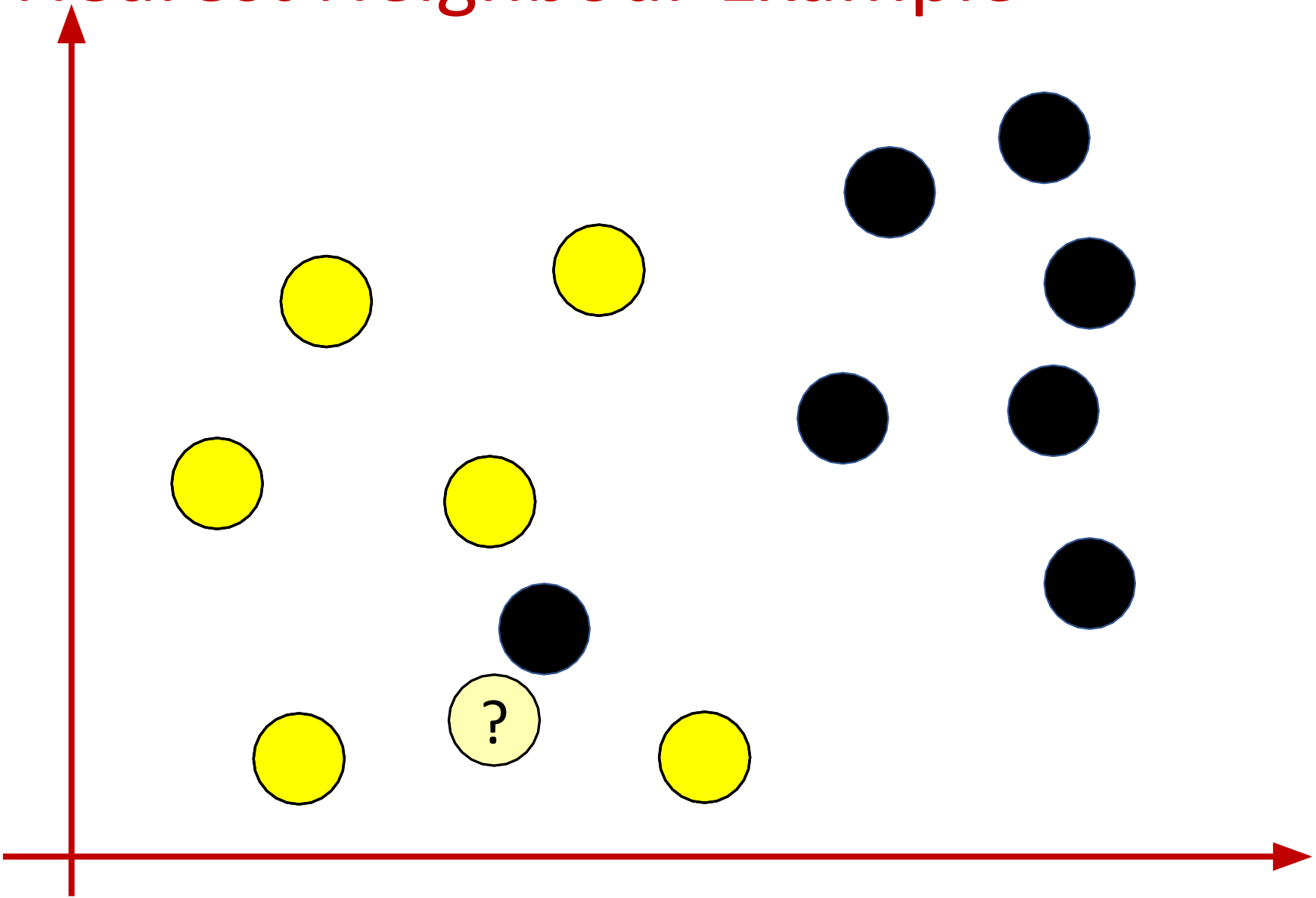
Nearest Neighbour Example



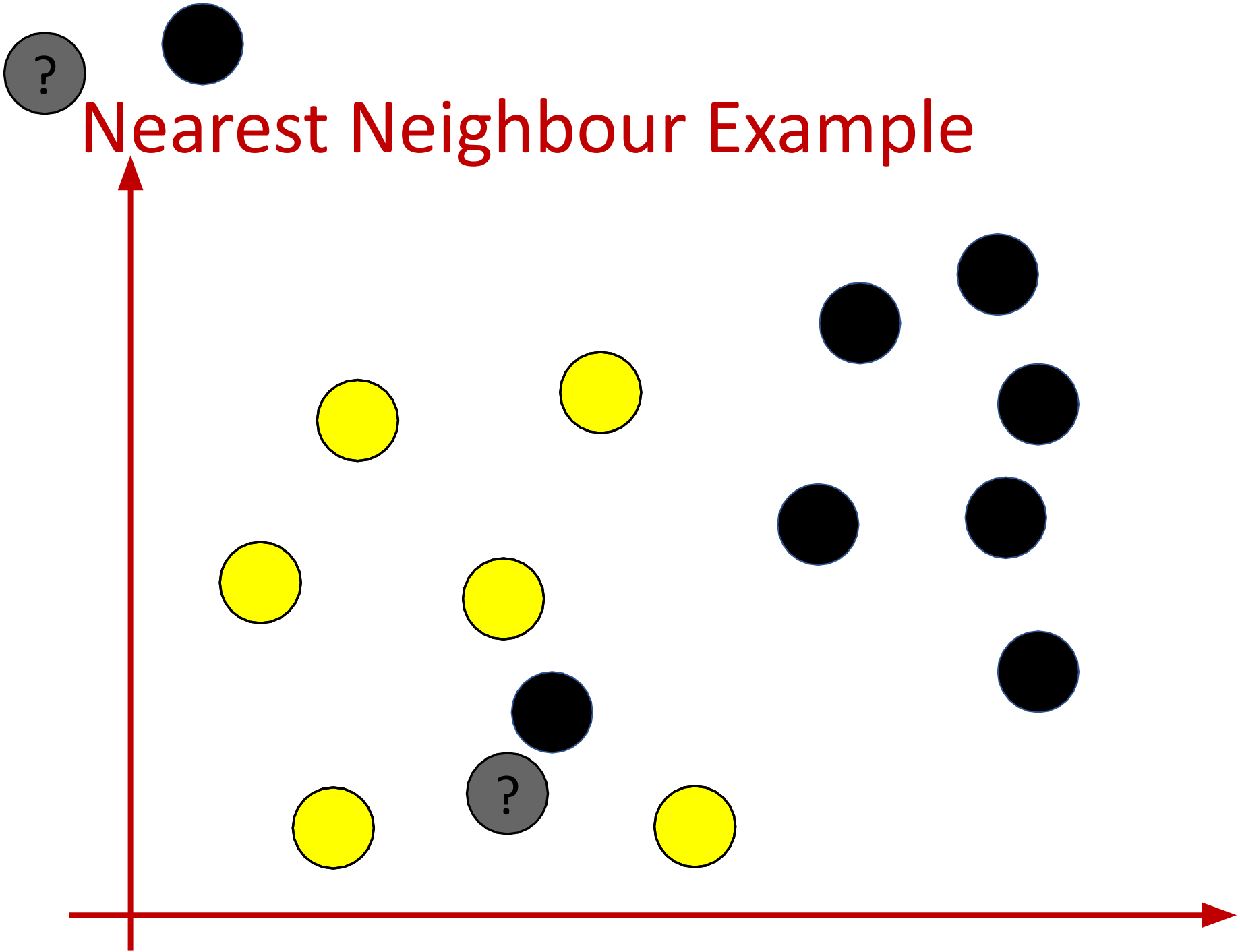
K-Nearest Neighbour

- A simple extension is to consider not just the nearest neighbour, but **several nearest neighbours**.
- This requires defining a neighbourhood; the standard approach is to use a neighbourhood that is just large enough to include a fixed number of points, k.
- Prediction is based on these k nearest neighbours.
  - If it is a classification problem then take a vote amongst the k-nearest neighbours
  - This approach is less sensitive to noise.

Nearest Neighbour Example



Nearest Neighbour Example



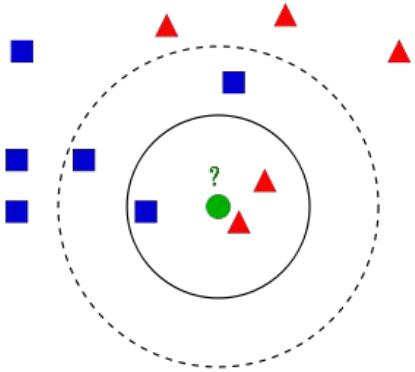
Nearest Neighbour Classifier

- Nearest Neighbour classification is mainly used when all the attribute values are continuous, although it can be modified to deal with categorical attributes
  - What is the problem with categorical variables?

- The idea is to estimate the classification of an unseen instance using the classification of the instance or instances that are closest to it, in some sense that we need to define

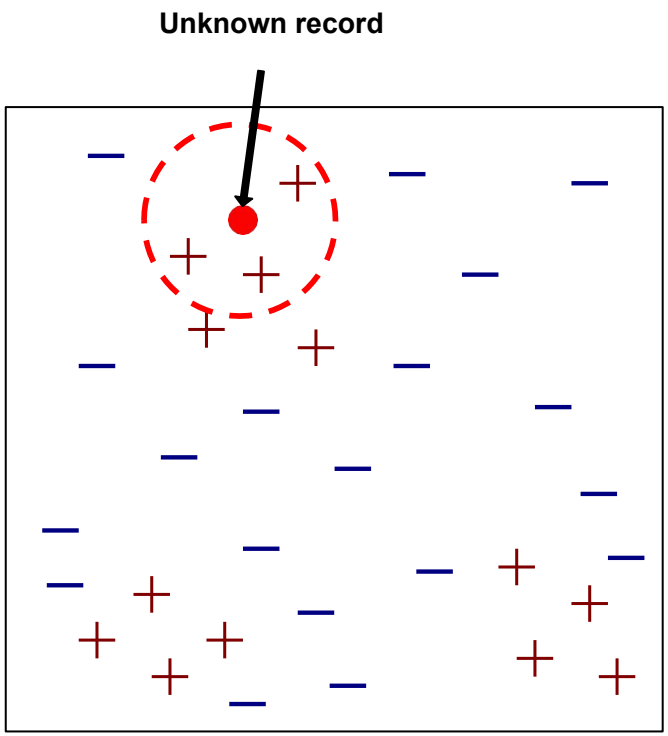
General Overview

- The nearest neighbour algorithm is a simple classification algorithm that classify a new point x according to its nearest neighbour class\label:
  - Let $X = < x_{\mathrm{i}}, b_{\mathrm{i}} >$ be a sample space of points and their classification. Given a new point y we find the $argmin_i \|x_i - y\|$ and classify y with $b_!$.
- An implicit assumptions is that close points have the same classification



Nearest-Neighbor Classifiers

- Requires three things
  - The set of stored records
  - Distance metric to compute distance between records
  - The value of k, the number of nearest neighbors to retrieve
- To classify an unknown record:
  - Compute distance to other training records
  - Identify k nearest neighbors
  - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)



Various methods of NN

- **<u>NN</u>** - Given a new point x, we wish to find it's nearest point and return it's classification.

- **<u>K-NN</u> -** Given a new point x, we wish to find it's k nearest points and return their average classification.

- **<u>Distance Weighted</u> -** Given a new point x , we assign weights to all the sample points according to the distance from x and classify x according to the weighted average.

Similarity and Distance

- One of the fundamental concepts of data mining is the notion of similarity between data points, often formulated as a numeric distance between data points.
- Similarity is the basis for many data mining procedures.
- We will consider the most direct use of similarity today.

kNN learning

- Assumes X = n-dim. space
  - discrete or continuous $f(x)$
- Let
  - $x = < a_1(x), \ldots, a_n(x) >$
  - $d(x_i, x_j)$ = Euclidean distance
- Algorithm (given new x)
  - find k nearest stored x_i (use $d(x, x_i)$)
  - take the most common value of $f(x_i)$

Similarity/Distance Between Data Items

- Each data item is represented with a set of attributes

"Closeness" is defined in terms of the distance (Euclidean or some other distance) between two data items.

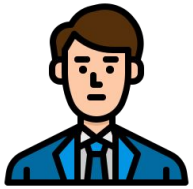
The Euclidean distance between $X_i = < a_1(X_i), \ldots, a_n(X_i)>$ and $X_j = < a_1(X_j), \ldots, a_n(X_j)>$ is defined as:

$$D(X_i, X_j) = \sqrt{\sum_{r=1}^{n} (a_r(X_i) - a_r(X_j))^2}$$

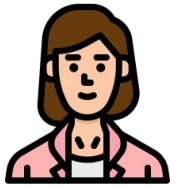
Similarity/Distance Between Data Items

- Distance measure

$$D(X_i, X_j) = \sqrt{\sum_{r=1}^{n} (a_r(X_i) - a_r(X_j))^2}$$



Bob:
Age = 35
Income = 35K
No. of credit cards = 3



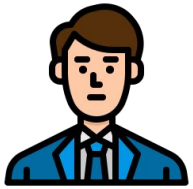
Alice:
Age = 22
Income = 50K
No. of credit cards = 2

$$D(Bob, Alice) = \sqrt{(35-22)^2 + (35K - 50K)^2 + (3-2)^2}$$

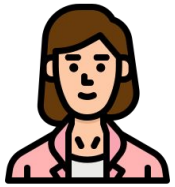
Similarity/Distance Between Data Items

- Distance measure

$$D(X_i, X_j) = \sqrt{\sum_{r=1}^{n} (a_r(X_i) - a_r(X_j))^2}$$



Bob:
Age = 35
Income = 35K
No. of credit cards = 3

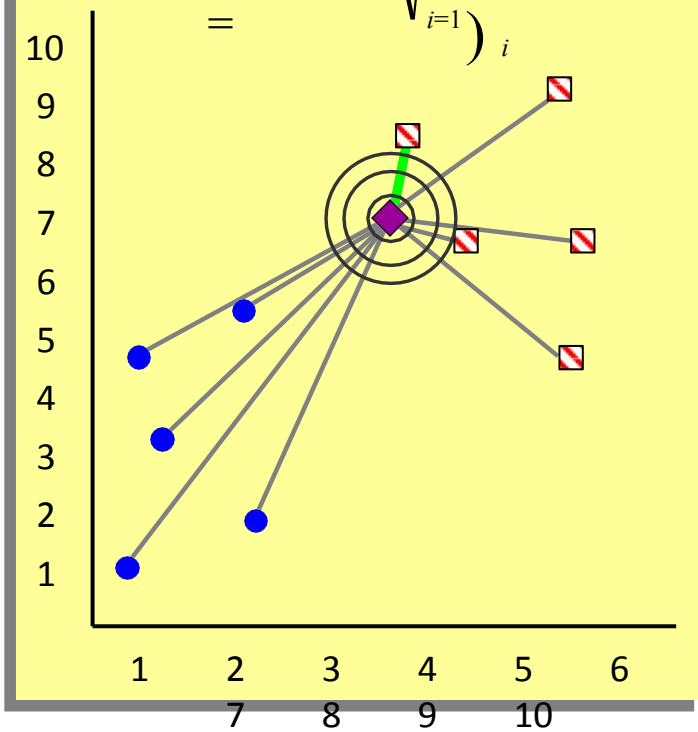


Alice:
Age = 22
Income = 50K
No. of credit cards = 2

$$D(Bob, Alice) = \sqrt{(35-22)^2 + (35K - 50K)^2 + (3-2)^2} = 15\,000.0056$$

Up to now we have assumed that the nearest neighbor algorithm uses the Euclidean Distance, however this need not be the case…

$$D(Q,C) = \sqrt{\sum_{i=1}^{n} (q_i - c_i)^2}$$



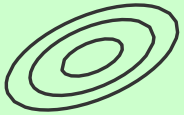
$$D(Q,C) = \sqrt[p]{\sum_{i=1}^{n} (q_i - c_i)^p}$$

Max (p=inf)

Manhattan (p=1)

Weighted Euclidean

Mahalanobis



Euclidean vs. Manhattan distance

Euclidean distance

Manhattan Distance Metric

- Manhattan distance measures distance parallel to each axis, not diagonally (in downtown Manhattan, to get from one point to another you generally walk North-South and East-West, rather than 'as the crow flies').

- In other words take the sum of the absolute values of the differences of the coordinates

$$D(X_i, X_j) = \sum_{r=1}^{n} \left| a_r(X_i) - a_r(X_j) \right|$$

Manhattan Distance Metric

- Lets assume we have a simple dataset containing three instances as follows. We are also given the query instance below. Calculate the Manhattan and Euclidean distance between the **first training example** and the **query**

Area	Weight	Height	Capacity
10	8	4	14
12	10	6	12
14	9	4	11

Area	Weight	Height	Capacity
5	4	4	10

Manhattan Distance Metric

```
import pandas as pd
# Compute the Manhattan and the Euclidian Distance
data = {'Area': [10, 12, 14],
        'Weight': [8, 10, 9],
        'Height': [4, 6, 4],
        'Capacity': [8, 12, 11]}
df = pd.DataFrame(data)

query = [5, 4, 4, 10]
df
```

Selecting Manhattan or Euclidean Distance Metric

- There are many proposed guidelines for which distance metric to adopt based on your task.

- However, as with many aspects of ML algorithms you should really empirically evaluate both metrics to determine which produces the best result for your algorithm. This process is known as hyper-parameter optimization and will be covered in a later lecture.

Let’s compute the distance between B and C for the following example:

ID	Outlook	Temp	Hum	Windy	Play?
A	sunny	hot	high	false	no
B	sunny	hot	high	true	no
C	overcast	hot	high	false	yes

Hamming Distance Metric

- Allows us to deal with problems that have features that are discrete rather than continuous-valued
- The value 0 is assigned for each feature where both cases have the same value, 1 for each where they are different.

$$D(X_i, X_j) = \sum_{r=1}^{n} \begin{cases} 1 & if \quad a_r(X_i) \,!= a_r(X_j) \\ 0 & if \quad a_r(X_i) == a_r(X_j) \end{cases}$$

ID	Outlook	Temp	Hum	Windy	Play?
A	sunny	hot	high	false	no
B	sunny	hot	high	true	no
C	overcast	hot	high	false	yes

What is distance between B and C?

Outlook (1) + Temp(0) + Hum(0) + Windy(1) = 2

Let's compute the distance between B and C for the following example:

ID	Outlook	Temp	Hum	Windy	Play?
A	sunny	hot	high	false	no
B	sunny	hot	high	true	no
C	overcast	hot	high	false	yes

What is distance between B and C?

Outlook (1) + Temp(0) + Hum(0) + Windy(1) = 2

Heterogeneous Distance Metric

- When we have a dataset that is a mix of discrete and continuous valued features we can combine distance measures such as Manhattan and Hamming.

- The Hamming metric is limited and provides limited information about the difference between features.

Domain Knowledge Distance Metric

- Humans can define **domain-specific** distance functions, for **discrete-valued non-numeric features**.
- In this way, they can incorporate their local **knowledge** (normally ordering for feature values) of the problem into the classification.
- Lets say we introduce a new attribute food with the following possible values ALL-BRAN CEREAL, APPLE PIE, BROCCOLI, CARROTS
  - Rather than using Hamming we could incorporate an order based on the level of calories in each of these foods.
  - A simple example is if we wished to express an ordering of these variables based on their total calories.
  - CARROTS < BROCCOLI < ALL-BRAN CEREAL < APPLE PIE

Domain Knowledge Distance Metric

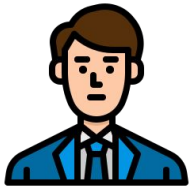
- We could assign the following values: CARROTS = 1, BROCCOLI =2, ALL-BRAN CEREAL=3, APPLE PIE = 4
  - dist(CARROTS, ALL-BRAN CEREAL) = |1-3| = 2
- Alternatively we might give them their actual calorie value. CARROTS = 30, BROCCOLI =50, ALL-BRAN CEREAL=70, APPLE PIE = 300
  - dist(CARROTS, ALL-BRAN CEREAL) = |30 - 70| = 40

- Clearly the above just has the effect of replacing a category attribute with a numerical one.

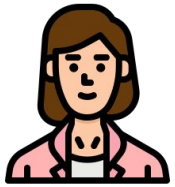
Similarity/Distance Between Data Items

- Distance measure

$$D(X_i, X_j) = \sqrt{\sum_{r=1}^{n} (a_r(X_i) - a_r(X_j))^2}$$



Bob:
Age = 35
Income = 35K
No. of credit cards = 3



Alice:
Age = 22
Income = 50K
No. of credit cards = 2

$$D(Bob, Alice) = \sqrt{(35-22)^2 + (35K - 50K)^2 + (3-2)^2}$$

Nearest Neighbours for Classification

- To determine the class of a new example E:
  - Calculate the distance between E and all examples in the training set
  - Select k examples closest to E in the training set
  - Assign E to the most common class (or some other combining function) among its k nearest neighbours

k-Nearest Neighbour Algorithms

a.k.a. Instance-based learning (IBL), Memory-based learning

- No model is built: Store all training examples
- Any processing is delayed until a new instance must be classified -> **lazy classification technique**

k-Nearest Neighbour Classifier
(k=3)

Customer	Age	Income (K)	No. cards	Response
John	35	35	3	Yes
Rachel	22	50	2	No
Ruth	63	200	1	No
Tom	59	170	1	No
Neil	25	40	4	Yes
David	**37**	**50**	**2**	**?**

Distance from David

k-Nearest Neighbour Classifier
(k=3)

Customer	Age	Income (K)	No. cards	Response
John	35	35	3	Yes
Rachel	22	50	2	No
Ruth	63	200	1	No
Tom	59	170	1	No
Neil	25	40	4	Yes
David	**37**	**50**	**2**	**?**

Distance from David
sqrt $[(35-37)^2+(35-50)^2+(3-2)^2]=$**15.16**
sqrt $[(22-37)^2+(50-50)^2+(2-2)^2]=$**15**
sqrt $[(63-37)^2+(200-50)^2+(1-2)^2]=$**152.23**
sqrt $[(59-37)^2+(170-50)^2+(1-2)^2]=$**122**
sqrt $[(25-37)^2+(40-50)^2+(4-2)^2]=$**15.74**

k-Nearest Neighbour Classifier
(k=3)

Customer	Age	Income (K)	No. cards	Response
John	35	35	3	Yes
Rachel	22	50	2	No
Ruth	63	200	1	No
Tom	59	170	1	No
Neil	25	40	4	Yes
David	**37**	**50**	**2**	**?**

Distance from David
sqrt $[(35-37)^2+(35-50)^2+(3-2)^2]=$**15.16**
sqrt $[(22-37)^2+(50-50)^2+(2-2)^2]=$**15**
sqrt $[(63-37)^2+(200-50)^2+(1-2)^2]=$**152.23**
sqrt $[(59-37)^2+(170-50)^2+(1-2)^2]=$**122**
sqrt $[(25-37)^2+(40-50)^2+(4-2)^2]=$**15.74**

k-Nearest Neighbour Classifier
(k=3)

Customer	Age	Income (K)	No. cards	Response	Distance from David
John	35	35	3	Yes	sqrt $[(35-37)^2+(35-50)^2+(3-2)^2]=$**15.16**
Rachel	22	50	2	No	sqrt $[(22-37)^2+(50-50)^2+(2-2)^2]=$**15**
Ruth	63	200	1	No	sqrt $[(63-37)^2+(200-50)^2+(1-2)^2]=$**152.23**
Tom	59	170	1	No	sqrt $[(59-37)^2+(170-50)^2+(1-2)^2]=$**122**
Neil	25	40	4	Yes	sqrt $[(25-37)^2+(40-50)^2+(4-2)^2]=$**15.74**
David	**37**	**50**	**2**	**Yes**	

Strengths and Weaknesses

- Strengths:
  - Simple to implement and use
  - Comprehensible – easy to explain prediction
  - Robust to noisy data by averaging k-nearest neighbours
  - Distance function can be tailored using domain knowledge
  - Can learn complex decision boundaries
    - Much more expressive than linear classifiers & decision trees
    - More on this later
- Weaknesses:
  - Need a lot of space to store all examples
  - Takes much more time to classify a new example than with a parsimonious model (need to compare distance to all other examples)
  - Distance function must be designed carefully with domain knowledge

Are These Similar at All?

- Humans would say “yes” although not perfectly so (both Homer).
- Nearest Neighbour method without carefully crafted features would say “no” since the colours and other superficial aspects are completely different. Need to focus on the shapes.

Are These Similar at All?

- Humans would say “yes” although not perfectly so (both Homer).
- Nearest Neighbour method without carefully crafted features would say “no” since the colours and other superficial aspects are completely different. Need to focus on the shapes.

Building a K Nearest Neighbour Classifier

- **Step 1. Read information from a dataset**
  - Read data from a dataset containing classified instances. Read each attribute of the dataset as well as corresponding class.

- **Step 2. Determine distance between each dataset entry and the query instance**
  - Use a suitable distance metric to calculate the distance between the query instance and all k neighbours

- **Step 3. Classify the query instance**
  - Identify k nearest data instances. Assign query instance category corresponding to most common category.

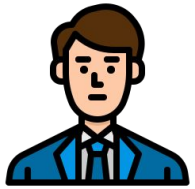
Problems Measuring Distance

- Since the performance of k-NN is strongly dependent on the choice of distance metric, you need to be aware of some pitfalls.

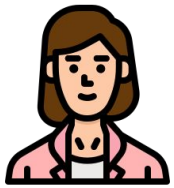
- The first problem arises when the **features are different** from each other; if one is weight measured in kg and the other is height measured in cm, it hardly makes sense to add them as would happen with Euclidian or Manhattan distance metrics. **The ranges of the variables will be different**.

- What is the main problem that arises from the above situation?

Similarity/Distance Between Data Items

- 

Bob:
Age = 35
Income = 35K
No. of credit cards = 3



Alice:
Age = 22
Income = 50K
No. of credit cards = 2

$$D(Bob, Alice) = \sqrt{(35 - 22)^2 + (35K - 50K)^2 + (3 - 2)^2} = 15\,000.0056$$

- Distance between neighbours can be dominated by an attribute with relatively large values (e.g., income in our example).
- Important to *normalize (e.g., map numbers to numbers* between 0-1)
  - Example: Income
  - Highest income = 500K
  - John's income is normalized to 95/500, Rachel's income is normalized to 215/500, etc.)

(there are more sophisticated ways to normalize)

Normalization

- Sometimes we need attributes to count equally
- Salaries: thousands of Euros
- Age: [0, 100]

- So you need to understand the properties and potential weaknesses of the method before using it
- Depending on the ML algorithm normalization can be useful or even required

Data Normalization

- Min-Max normalization
  - For an attribue *A* we have values in range $[min_A, max_A]$
  - We want to scale them in [0,1]

The new values can be computed as:

$$v' = \frac{v - min_A}{max_A - min_A}$$

Min-Max scaling

Min income = 35 Max income = 200

Customer	Age	Income (K)	No. cards
John	35	35	3
Rachel	22	50	2
Ruth	63	200	1
Tom	59	170	1
Neil	25	40	4
David	37	50	2

Scaled Income

Min-Max scaling

Min income = 35 Max income = 200

Customer	Age	Income (K)	No. cards
John	35	35	3
Rachel	22	50	2
Ruth	63	200	1
Tom	59	170	1
Neil	25	40	4
David	37	50	2

Scaled Income
(35-35)/(200-35) = **0**
(50-35)/(200-35) = **0.09**
(200-35)/(200-35) = **1**
(170-35)/(200-35) = **0.81**
(40-35)/(200-35) = **0.03**
(50-35)/(200-35) = **0.09**

Min-Max scaling

- It is possible to scale to new values of min and max
- Suppose that we want a range $[new_min_A, new_max_A]$

The new values can be computed as:

$$v' = \frac{v - min_A}{max_A - min_A}(new_max_A - new_min_A) + new_min_A$$

Normalising Data

- Normalization is the rescaling of the features into the range between **0 and 1**.
- This can improve the performance for algorithms that assign a weight to features such as linear regression and in particular for algorithms that utilize geometric distance such as KNNs.
- In the following dataset I print out the first few rows of the feature data. You will notice that the second feature has a much greater range than any of the other features.

Normalising Data

- Scikit allows us to normalize all features using the **MinMaxScaler** class in **sklearn.preprocessing**.
- Note the **fit_transform** function takes as input a NumPy array or a DataFrame.

Normalising Data

```
from sklearn import preprocessing
import numpy as np
customer = np.array([[ 35, 35, 3],
                     [ 22, 50, 2],
                     [ 63, 200, 1],
                     [ 59, 170, 1],
                     [ 25, 40, 4]]
                     )
print(customer)

scalingObj = preprocessing.MinMaxScaler()
new_customer = scalingObj.fit_transform(customer)
print(new_customer)
```

Min-Max scaling

Min income = 35 Max income = 200

Customer	Age	Income (K)	No. cards
John	35	35	3
Rachel	22	50	2
Ruth	63	200	1
Tom	59	170	1
Neil	25	40	4
David	37	50	2

Scaled Income
(35-35)/(200-35) = **0**
(50-35)/(200-35) = **0.09**
(200-35)/(200-35) = **1**
(170-35)/(200-35) = **0.81**
(40-35)/(200-35) = **0.03**
(50-35)/(200-35) = **0.09**

Normalising Data

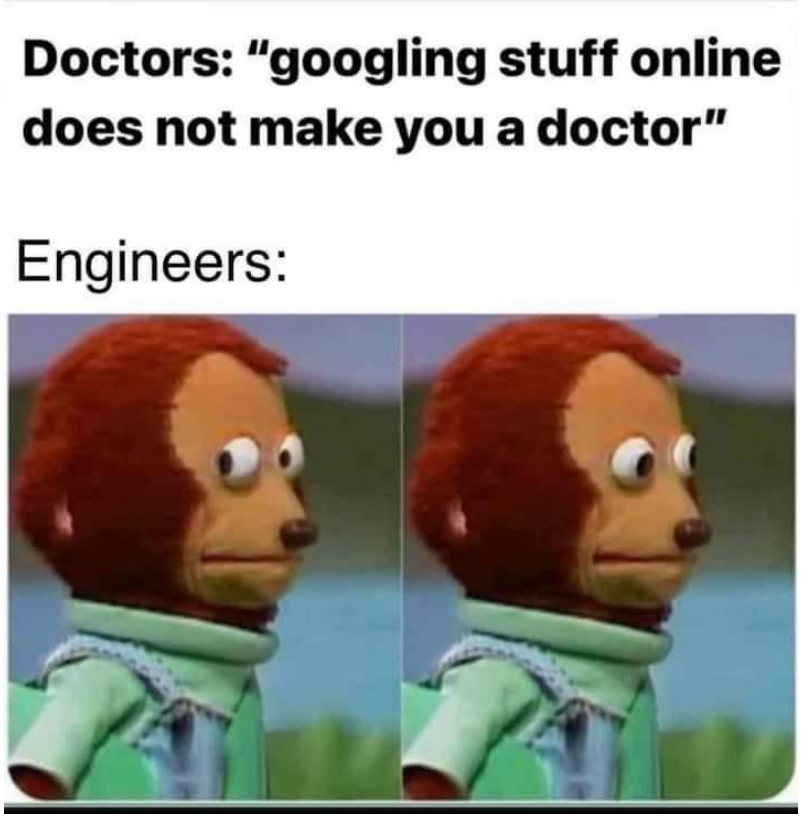
```
# If a new possible customer is coming:
david = np.array([[37, 50, 2]])
new_david = scalingObj.transform(david)
new_david
```

Normalising Data

```
# We can check the attributes of the Scaler
print(scalingObj.data_min_)
print(scalingObj.data_max_)
```

Normalising Data

```
# We can check the attributes of the Scaler
print(scalingObj.data_min_)
print(scalingObj.data_max_)
```



Normalising Data - DataFrame

```
# let’s try to scale a DataFrame
import pandas as pd
from sklearn import preprocessing
import numpy as np

np.random.seed(2021)
seriesA = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesB = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesC = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])

df = pd.DataFrame({'one' : seriesA, 'two' : seriesB, 'three' : seriesC})
print(df)

scalingObj = preprocessing.MinMaxScaler()
df[['one', 'two']]= scalingObj.fit_transform( df[['one', 'two']] )
print(df)
```

Normalising Data - DataFrame

```
# let's try to scale a DataFrame
import pandas as pd
from sklearn import preprocessing
import numpy as np

np.random.seed(2021)
seriesA = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesB = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesC = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])

df = pd.DataFrame({'one' : seriesA, 'two' : seriesB, 'three' : seriesC})
print(df)

scalingObj = preprocessin
df[['one', 'two']]= scali
print(df)
```

```
          one        two      three
a  60.597828  99.724328  66.216051
b  73.336936  12.816238  78.431013
c  13.894716  17.899311   9.689440
d  31.267308  75.292543   5.857129
        one       two      three
a  0.785689  1.000000  66.216051
b  1.000000  0.000000  78.431013
c  0.000000  0.058488   9.689440
d  0.292260  0.718878   5.857129
```

Standardizing Data

- Z-score normalization
  - For an attribue A we compute the mean and the standard deviation

- The new values can be computed as:

$$v' = \frac{v - mean_A}{standard_deviation_A}$$

Standardizing Data

Mean income = 99 Standard dev = 71

```
std = np.std([35,50,200,170,40])
```

Customer	Age	Income (K)	No. cards
John	35	35	3
Rachel	22	50	2
Ruth	63	200	1
Tom	59	170	1
Neil	25	40	4
David	37	50	2

Scaled Income

Standardizing Data

Mean income = 99 Standard dev = 71

```
std = np.std([35,50,200,170,40])
```

Customer	Age	Income (K)	No. cards
John	35	35	3
Rachel	22	50	2
Ruth	63	200	1
Tom	59	170	1
Neil	25	40	4
David	37	50	2

Scaled Income
(35-99)/71= **-0.90**
(50-99)/71= **-0.69**
(200-99)/71= **1.42**
(170-99)/71= **1.00**
(40-99)/71= **-0.83**

Standardizing Data

```
from sklearn import preprocessing
import numpy as np
customer = np.array([[ 35, 35, 3],
                     [ 22, 50, 2],
                     [ 63, 200, 1],
                     [ 59, 170, 1],
                     [ 25, 40, 4]]
                     )
print(customer)

scalingObj = preprocessing.StandardScaler()
new_customer = scalingObj.fit_transform(customer)
print(new_customer)
```

Standardizing Data

```
from sklearn import preprocessing
import numpy as np
customer = np.array([[ 35, 35, 3],
                     [ 22, 50, 2],
                     [ 63, 200, 1],

print(customer)

scalingObj = prepro
new_customer = scal
print(new_customer)
```

```
[[ 35  35   3]
 [ 22  50   2]
 [ 63 200   1]
 [ 59 170   1]
 [ 25  40   4]]
[[-0.33932637 -0.90114035  0.68599434]
 [-1.09988549 -0.68993558 -0.17149859]
 [ 1.29880095  1.42211211 -1.02899151]
 [ 1.06478276  0.99970257 -1.02899151]
 [-0.92437184 -0.83073876  1.54348727]]
```

Standardizing Data

```
# If a new possible customer is coming:
david = np.array([[37, 50, 2]])
new_david = scalingObj.transform(david)
new_david
```

Standardizing Data

```
# We can check the attributes of the Scaler
print(scalingObj.mean_)
print(scalingObj.var_)
```

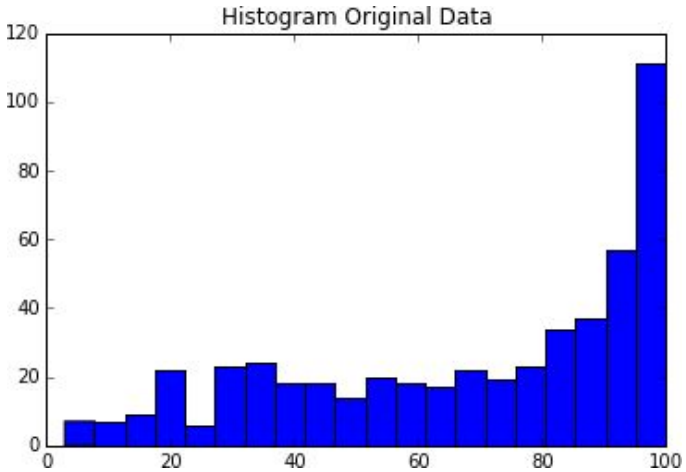
Standardizing Data - DataFrame

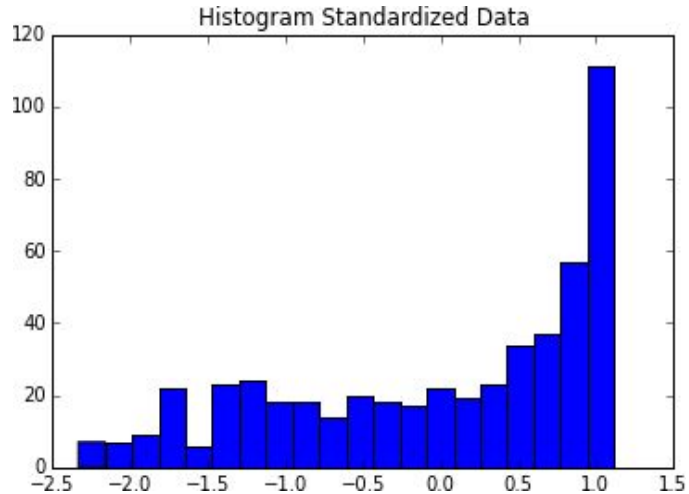
```
# let’s try to scale a DataFrame
import pandas as pd
from sklearn import preprocessing
import numpy as np

np.random.seed(2021)
seriesA = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesB = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesC = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])

df = pd.DataFrame({'one' : seriesA, 'two' : seriesB, 'three' : seriesC})
print(df)

scalingObj = preprocessing.StandardScaler()
df[['one', 'two']]= scalingObj.fit_transform( df[['one', 'two']] )
print(df)
```

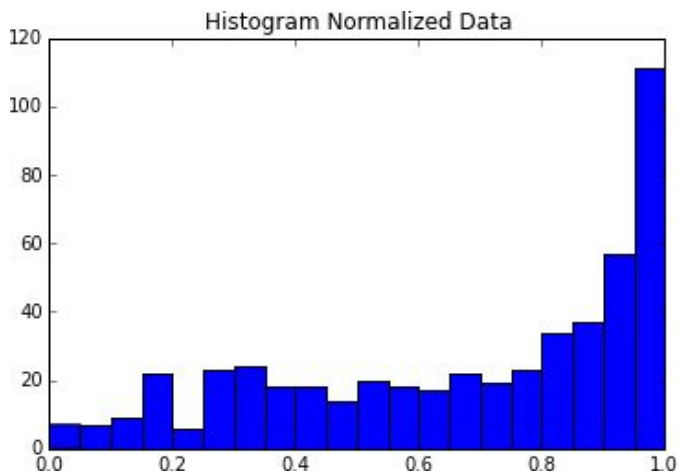
Standardizing Data - DataFrame

```
# let's try to scale a DataFrame
import pandas as pd
from sklearn import preprocessing
import numpy as np

np.random.seed(2021)
seriesA = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesB = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])
seriesC = pd.Series(np.random.rand(4)*100, index=['a', 'b', 'c', 'd'])

df = pd.DataFrame({'one' : seriesA
print(df)

scalingObj = preprocessing.Standa:
df[['one', 'two']]= scalingObj.fit
print(df)
```

```
          one        two      three
a  60.597828  99.724328  66.216051
b  73.336936  12.816238  78.431013
c  13.894716  17.899311   9.689440
d  31.267308  75.292543   5.857129
        one       two      three
a  0.674385  1.300298  66.216051
b  1.217312 -1.039805  78.431013
c -1.316049 -0.902937   9.689440
d -0.575648  0.642444   5.857129
```







- Unless you fully understand in-depth all the ML algorithms you use it can be very difficult to know if you should use normalization or standardization when scaling your data.

- It is generally recommended that try both approaches to determine which provides the best accuracy value.

Problems Measuring Distance – Irrelevant Features

- The normal Euclidean distance formula assumes that all attributes are of equal importance.
- The **weights allow different attributes to have different importance** in the computation of the overall distance.

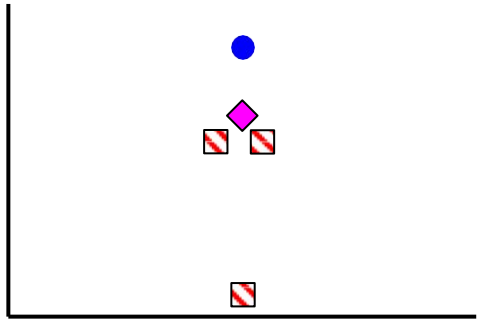
The weighted Euclidean distance would then look like this:

$$D(X_i, X_j) = \sqrt{\sum_{r=1}^{n} w_r (a_r(X_i) - a_r(X_j))^2}$$

- Where w_r is the weight of attribute *r*

Units of measurement

- The nearest neighbour algorithm is sensitive to the units of measurement

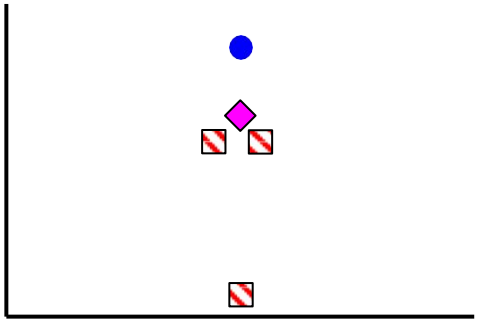


X axis measured in **centimeters**
Y axis measure in dollars

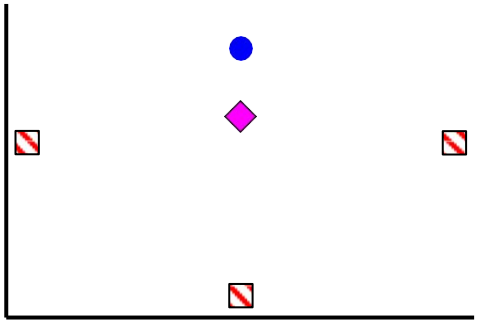
The nearest neighbour to the **pink** unknown instance is **red**.

Units of measurement

- The nearest neighbour algorithm is sensitive to the units of measurement



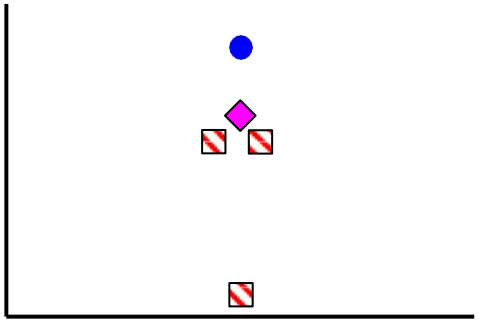
X axis measured in **centimeters**
Y axis measure in dollars
The nearest neighbour to the **pink** unknown instance is **red**.



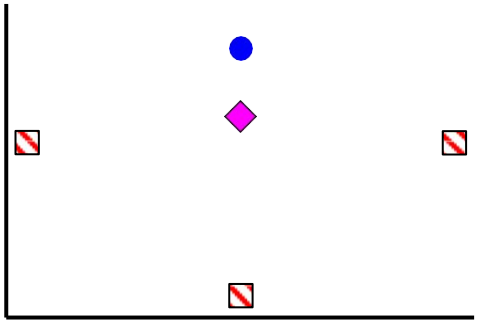
X axis measured in **millimeters**
Y axis measure in dollars
The nearest neighbour to the **pink** unknown instance is **blue**.

Units of measurement

- One solution is to normalize the units to pure numbers. Typically the features are Z- scaled to have a mean of zero and a standard deviation of one.



X axis measured in **centimeters**
Y axis measure in dollars
The nearest neighbour to the **pink** unknown instance is **red**.



X axis measured in **millimeters**
Y axis measure in dollars
The nearest neighbour to the **pink** unknown instance is **blue**.

Normalization vs Scaling

- Fundamental part of machine learning.
- Even if you have great data, one attribute might shadow the others.
- Normalization solves these problems.

- **Min-max normalization**: All attributes will have the exact same range. Sensible to outliers.
- **Z-score scaling**: Handles outliers, scaled data have different ranges.

Problems Measuring Distance – Irrelevant Features

- The other principal problem is that **all features are included equally in the calculations** for any of the four metrics that we have seen, even though some features may be **redundant** or **less relevant**.
- Therefore, a number of features may skew the result even through they might of little or no impact to the classification.

Problems Measuring Distance – Irrelevant Features

- Solution A:
  - **Assign weights** to each dimension (Optimise weighting to minimise error)
- Solution B:
  - Give some dimensions **0 weight** (Feature subset solution)
- Either way, since we cannot know in advance what weighting to give dimensions, systematic repeated experiments are needed to optimise them.
- We will look feature selection in more detail later in the module.

Quick game

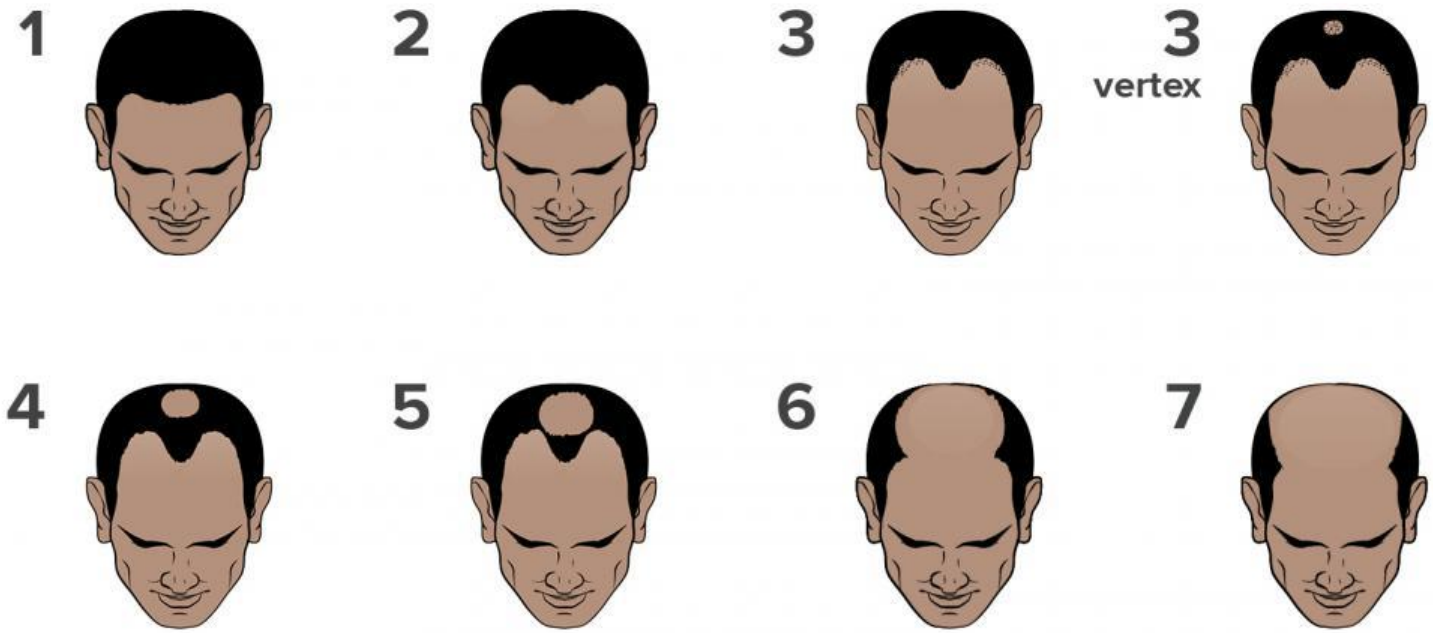
- How old am I?

Quick game

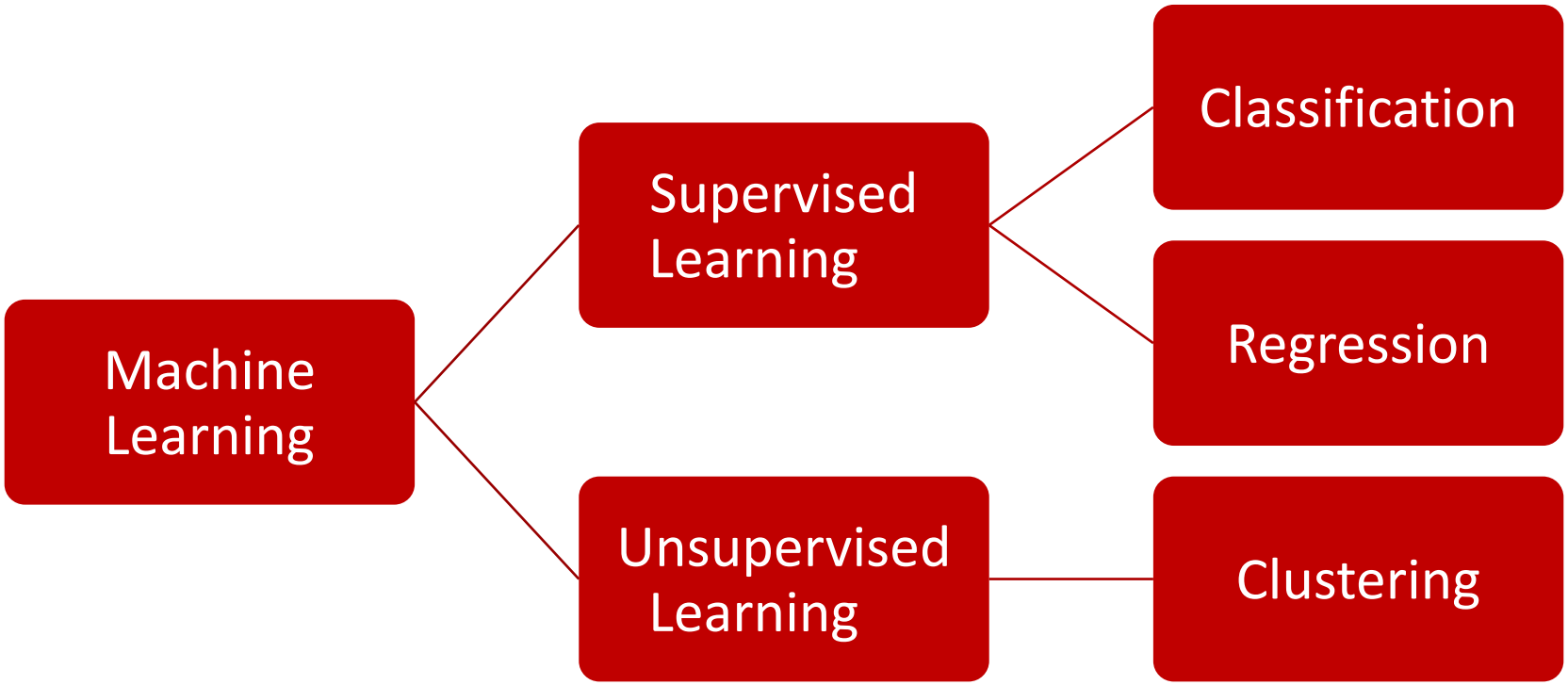
- Which variables did you consider?

MEDICALNEWSTODAY

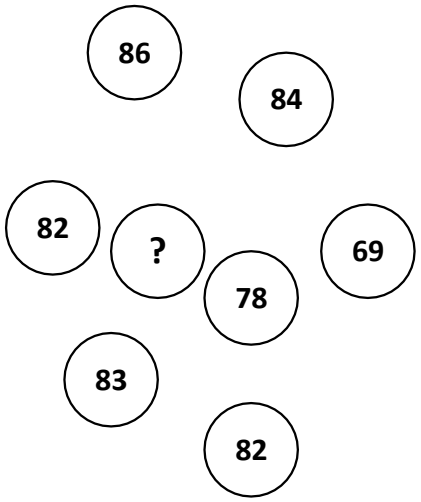
Norwood Scale



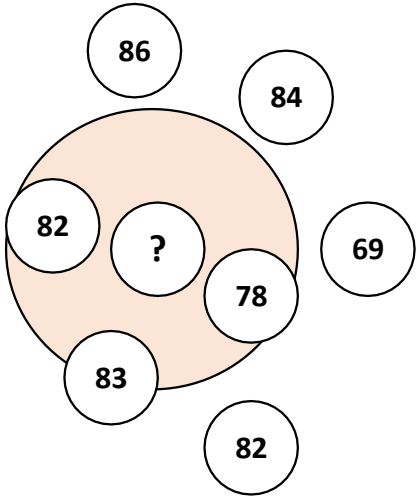
Algorithm Hierarchy



kNN - Regression

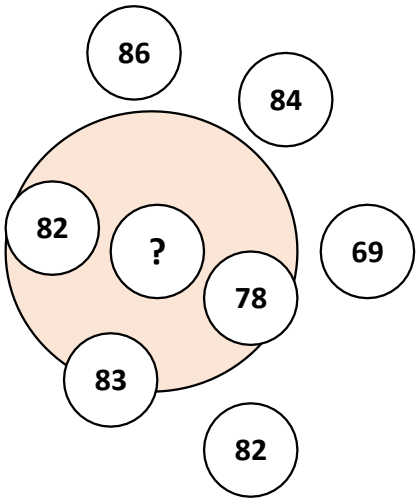


kNN - Regression



kNN - Regression

- kNN can be applied to regression problems.
- The answer is the average value of k of the neighbours.
- What is the answer if: k = 3?
  - (82+78+83)/3 = 81.



Mean Absolute Error

$$MAE = \frac{\sum_{i=1}^{n} |y_i - \widehat{y_i}|}{n}$$

•

Where:

- y_i is the real value
- $\widehat{y_i}$ is the predicted value
- n is the number of elements

Mean Squared Error

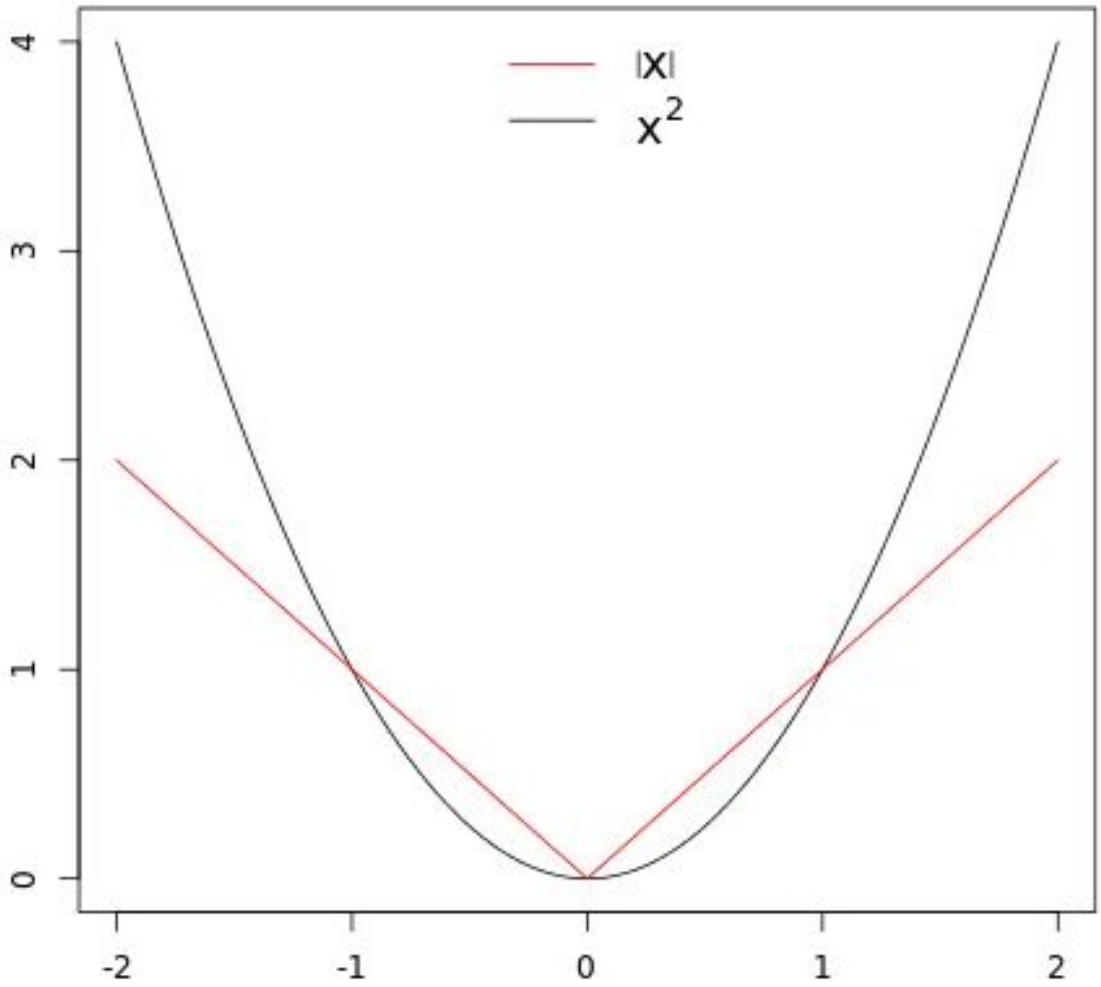
$$MSE = \frac{\sum_{i=1}^{n}(y_i - \widehat{y_i})^2}{n}$$

•

Where:

- y_i is the real value
- $\widehat{y_i}$ is the predicted value
- n is the number of elements

MAE vs MSE



MAE vs MSE

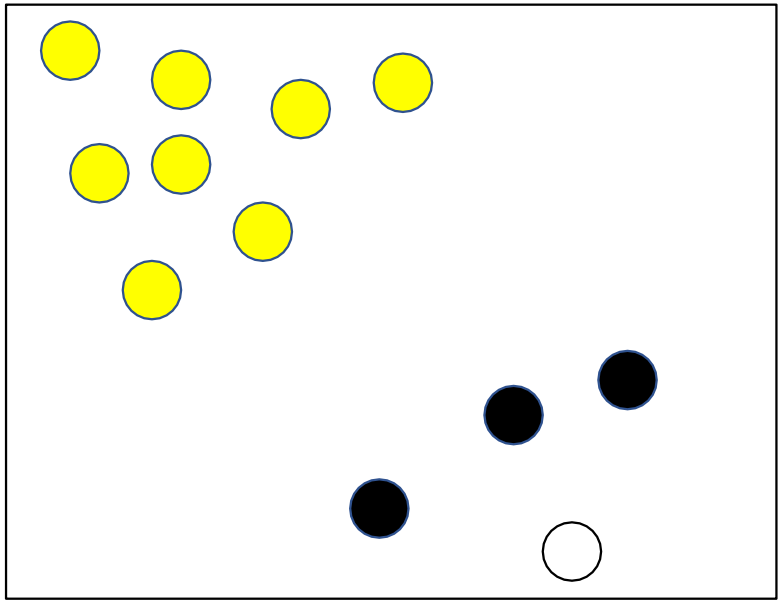
Both good because:

- No negative error values and mutual cancellation of errors.

- MSE emphasizes more high errors. Less affected by smaller errors.
- MAE fair on all mistakes.

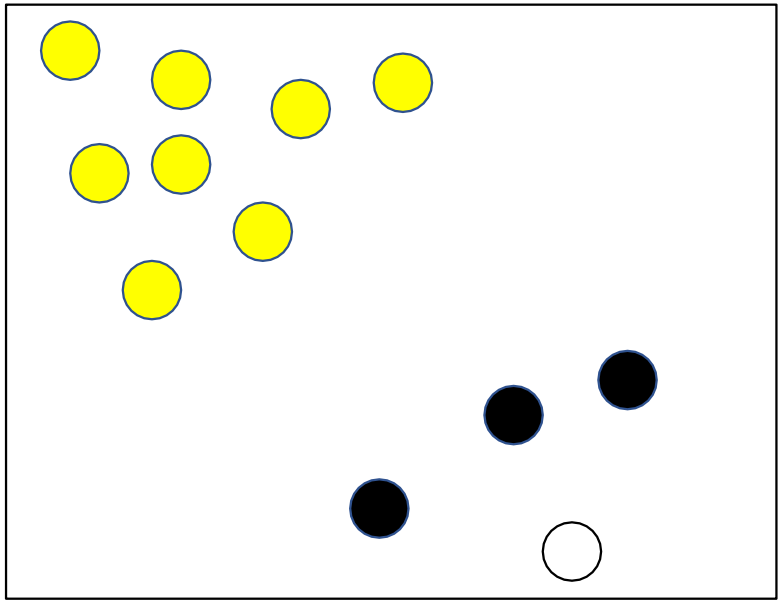
Problem with KNN

- Assume in the following example that we are using **k=7**?
- What are the potential problems?
- How can we address the problem?



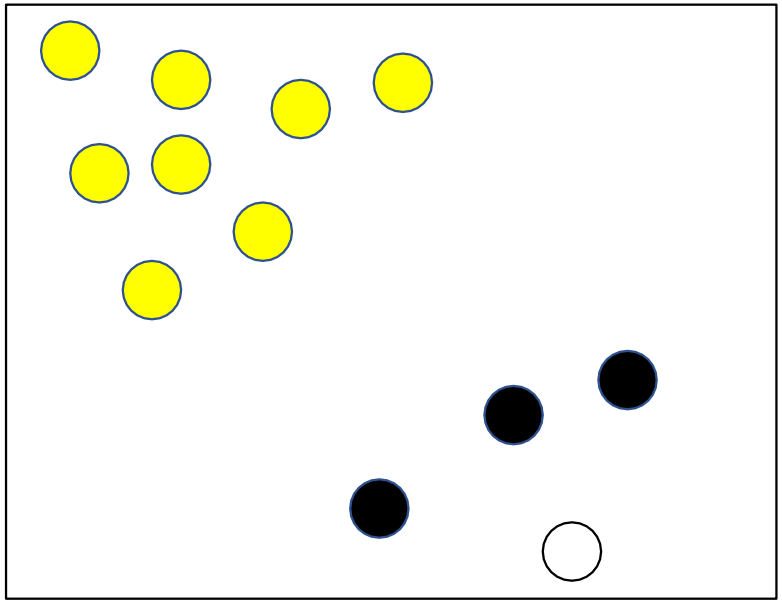
Problem with KNN

- **Distance-Weighted kNN**
  - Give each neighbour weight: inverse of distance from target



Distance Weight kNN

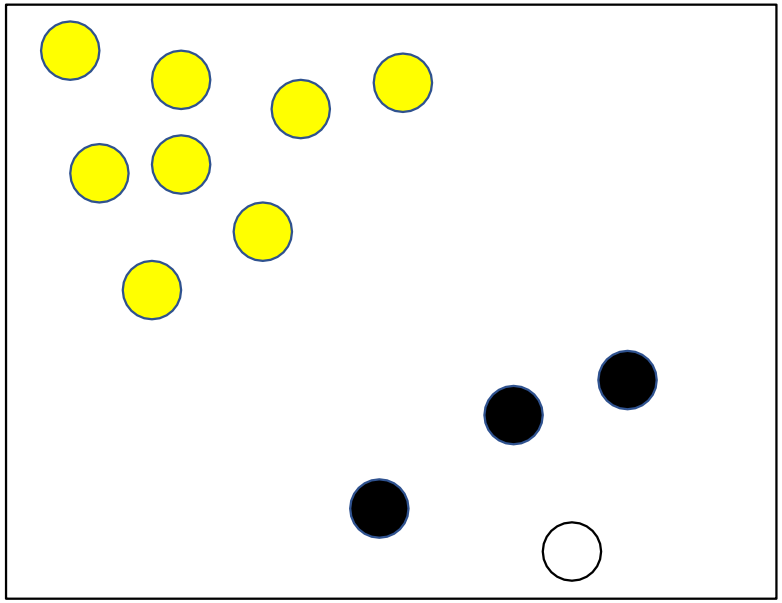
- It is the vote of each neighbour that is weighted according to how close it is to the target, so that **closer neighbour's influence the prediction more**.
- Interestingly, we can then use all cases as neighbours, since those very far from the target will have little influence on the prediction, but none will be completely ignored.



Distance Weight kNN

- Where $w_j = \frac{1}{D(X_i, X_j)^2}$

- The weight is on the final decision, not on the distance!



Distance Weight kNN

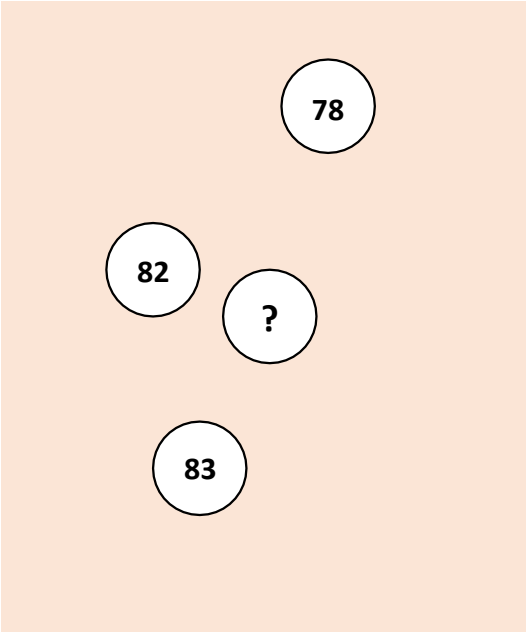
- $$f(X_q) = \frac{\sum_{i=1}^{k} w_j f(X_i)}{\sum_{i=1}^{k} w_j}$$

- Where $w_j = \frac{1}{D(X_i, X_j)^2}$

Distance Weighted k-NN Algorithm Regression Example

- Consider the basic regression example depicted in the slide.
- Distance between query instance and:
  - Case 82 is 2
  - Case 83 is 4
  - Case 78 is 6

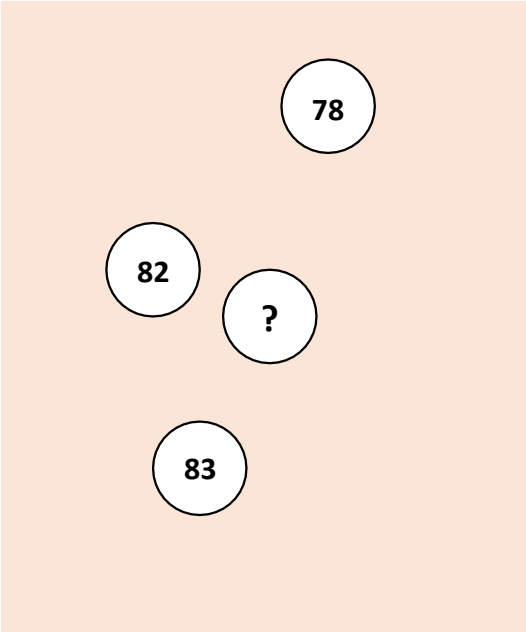
- What is the value of the query instance.



Distance Weighted k-NN Algorithm Regression Example

- Consider the basic regression example depicted in the slide.
- Distance between query instance and:
  - Case 82 is 2
  - Case 83 is 4
  - Case 78 is 6

- What is the value of the query instance.



Distance Weighted k-NN (Classification)

- For classification it is the vote of each neighbour that is weighted according to how close it is to the target, so that closer neighbour's influence the prediction more.
- Again we can then use all cases as neighbours, since those very far from the target will have little influence on the prediction, but none will be completely ignored.

Do the example

Customer	Age	Income (K)	No. cards	Response
John	35	35	3	Yes
Rachel	22	50	2	No
Ruth	63	200	1	No
Tom	59	170	1	No
Neil	25	40	4	Yes
David	**37**	**50**	**2**	**?**

Distance from David
sqrt $[(35-37)^2+(35-50)^2+(3-2)^2]=$**15.16**
sqrt $[(22-37)^2+(50-50)^2+(2-2)^2]=$**15**
sqrt $[(63-37)^2+(200-50)^2+(1-2)^2]=$**152.23**
sqrt $[(59-37)^2+(170-50)^2+(1-2)^2]=$**122**
sqrt $[(25-37)^2+(40-50)^2+(4-2)^2]=$**15.74**

Eager vs. Lazy Learner

- Eager Learning (Such as decision trees, neural networks, Bayesian classifiers)
  - When given training data, it **constructs a model** for future use in prediction that summarises the data
  - **Slow in model construction**, quicker in subsequent use
  - Model itself may be useful/informative

- Instance based learning often referred to as lazy Learning
  - **No** explicit global **model** constructed
  - **Calculations deferred** until new case to be classified
  - **Creates many local approximations**, whereas eager learners create a global approximation
  - **Significant calculations** needed to take place for each new query (can be slow)

When to use k-Nearest Neighbour

- Consider using when:
  - Moderate number of training cases
  - Moderate number of features per instance (< 20)

- Primary Benefits
  - **Comprehensibility**: easy to understand
  - Relatively straight-forward to **implement**
  - Effective classifiers **for complex target functions** => good for diverse concepts
  - Can be used for **both regression and classification** problems and can mix feature types in the one dataset so they are very flexible

When to use k-Nearest Neighbour

- Drawbacks:
  - Problems with **irrelevant features**. Can require substantial resources to identify.
  - Can be very sensitive to imbalanced datasets
  - It is called instance-based because it constructs hypotheses directly from the training instances themselves. This means that the hypothesis complexity can grow with the data. Therefore it can be **slow in classification/regression if there is a large dataset.**

Curse of Dimensionality

- After overfitting, the biggest problem in machine learning is the “curse of dimensionality”.
- The curse of dimensionality refers to various issues that arise when **analyzing data in high-dimensional spaces** (potentially with hundreds or even thousands of dimensions) and is particularly relevant to instance based learners.
- The common root of these difficulties lies in the fact that **as dimensionality increases, the volume of space increases so fast that the available data becomes sparse.**
  - This raises particular problems for ML techniques that rely on detecting areas where objects form groups with similar properties.

k-d trees

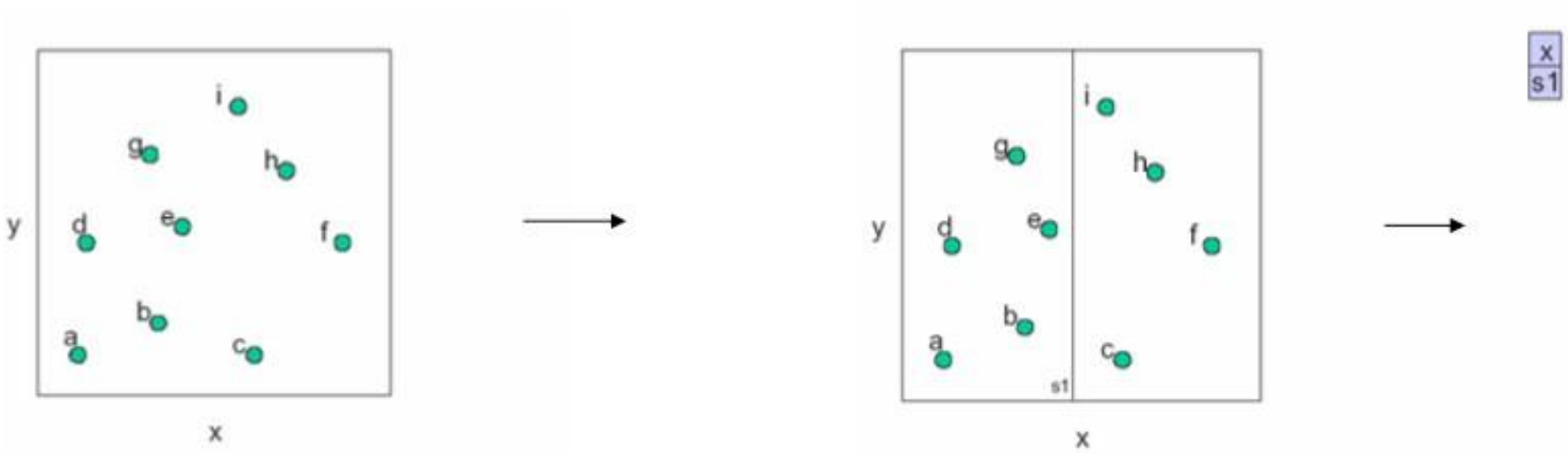
k-d trees

- Jon Bentley, 1975
- Tree used to store spatial data (quite popular in databases)
- k-d trees are guaranteed log2 n depth where n is the number of points in the set
  - Store points in d-dimensional space (equivalent to vectors in d dimensional space)

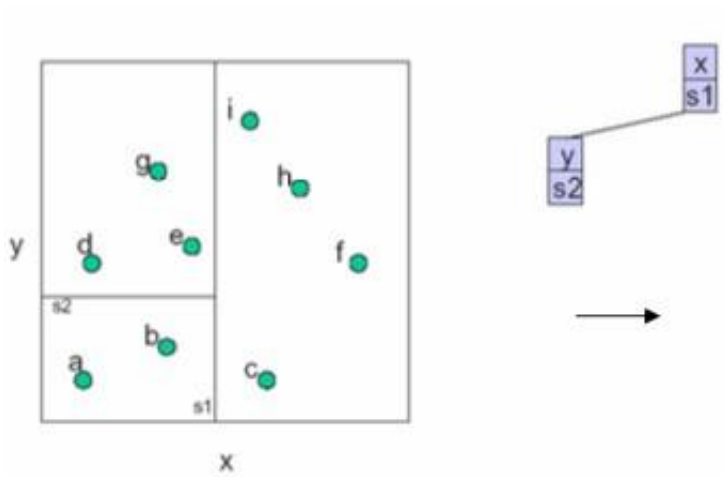
K-d trees

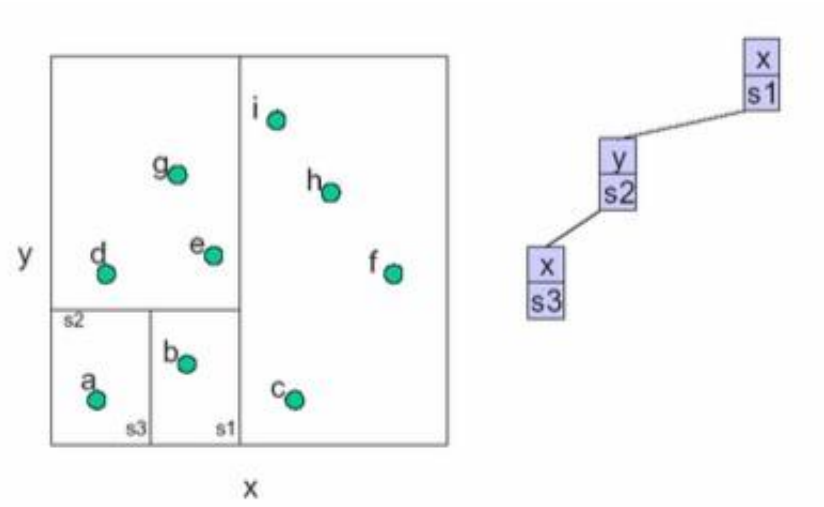
- If there is just one point, form a leaf with that point
- Otherwise, divide the points in half by a line perpendicular to one of the axis
- Recursive construct k-d tree for the two sets of points
- Division:
  - Divide points perpendicular to the axis with widest spread
  - Divide in a round-robin fashion

Construction

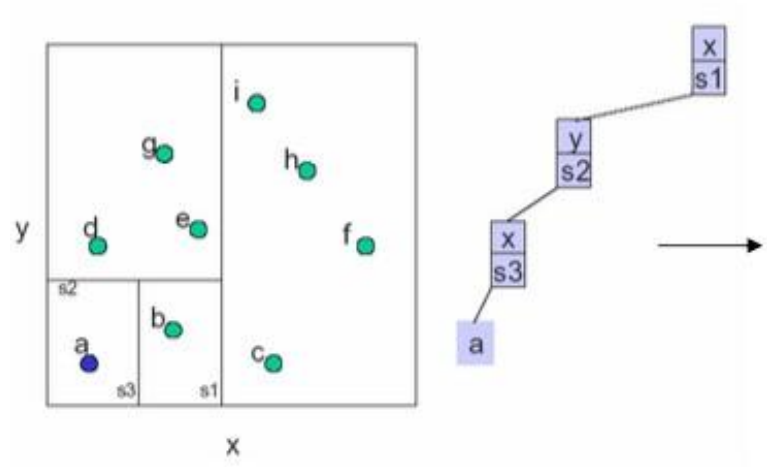


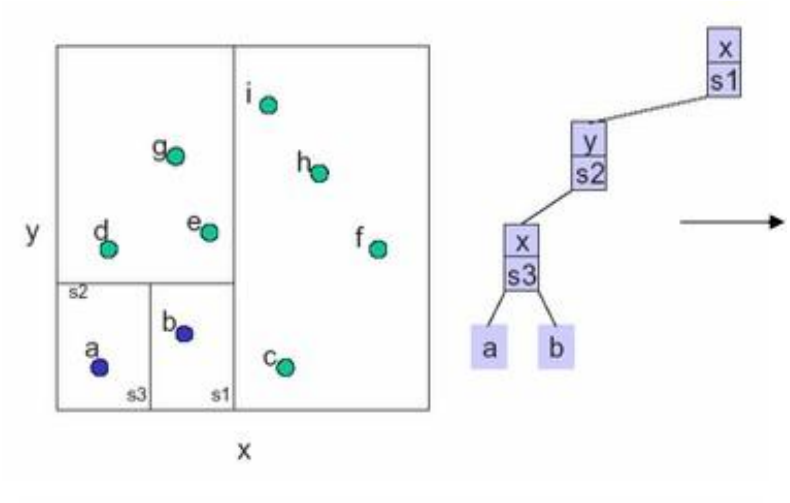
Construction



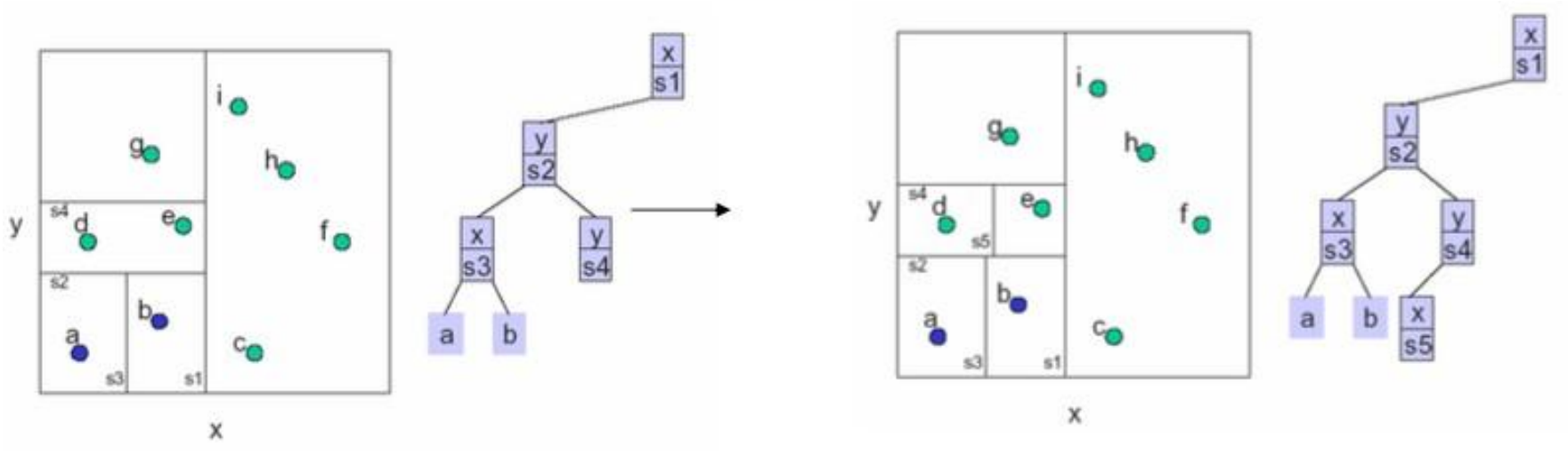


Construction

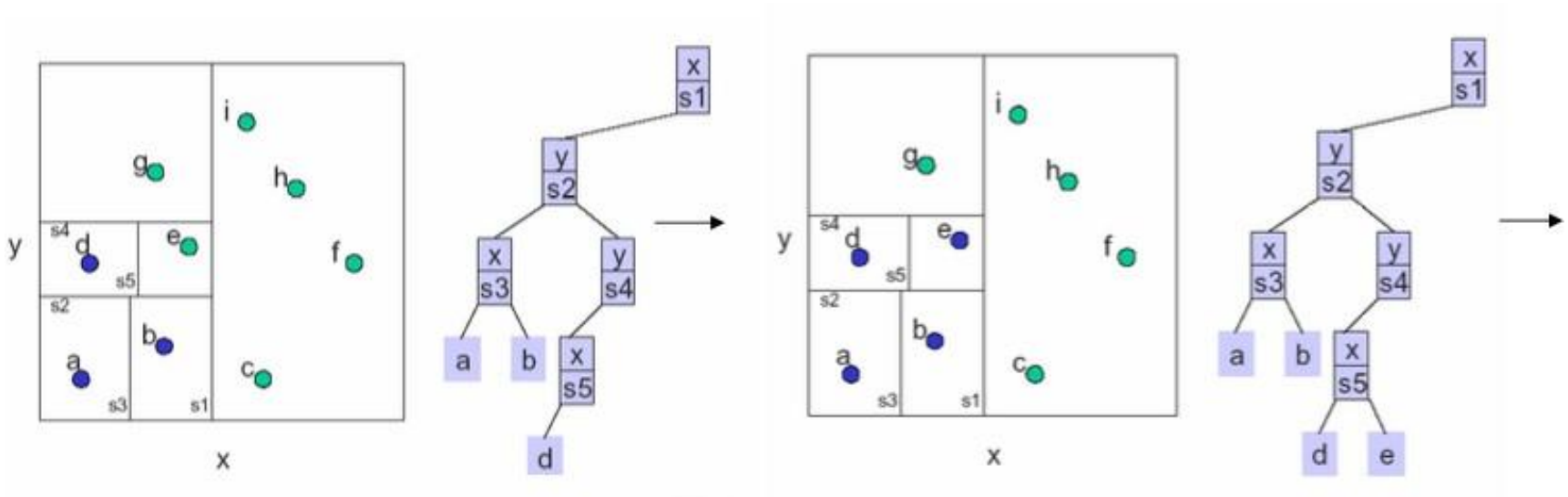




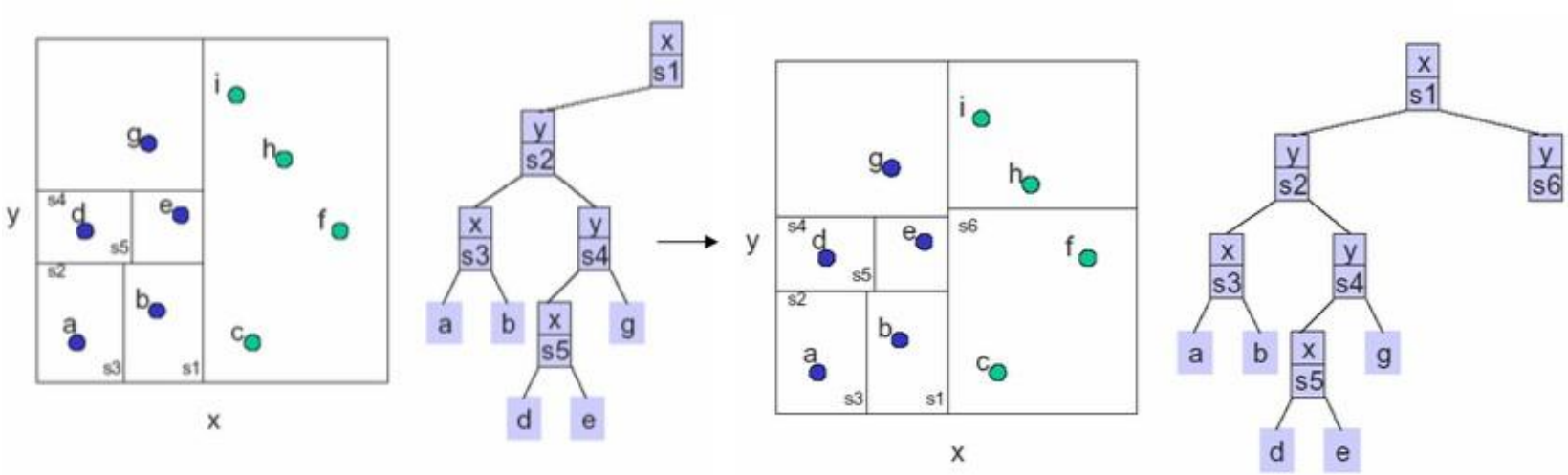
Construction



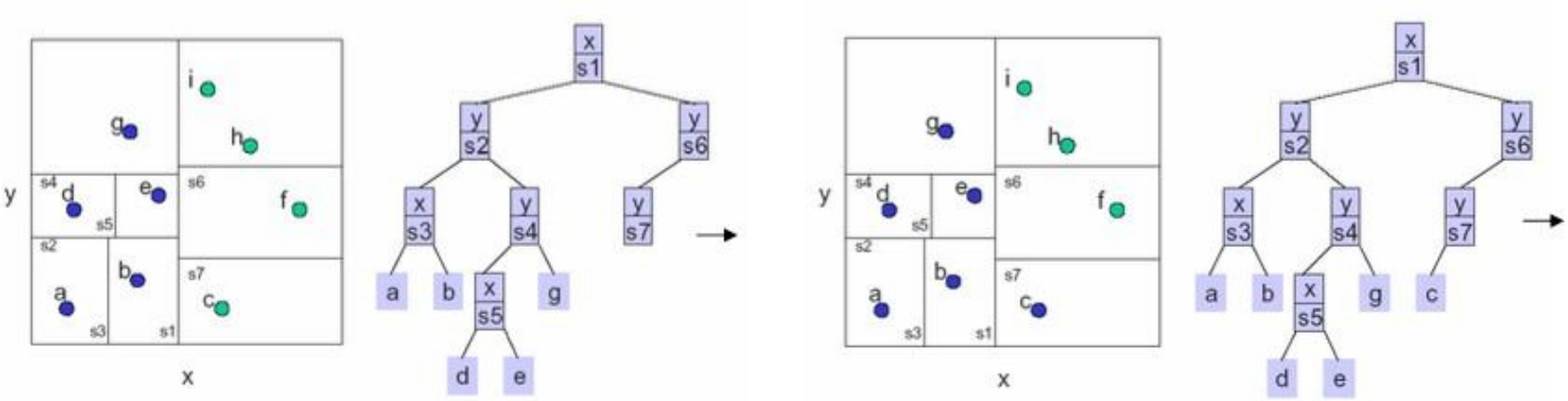
Construction



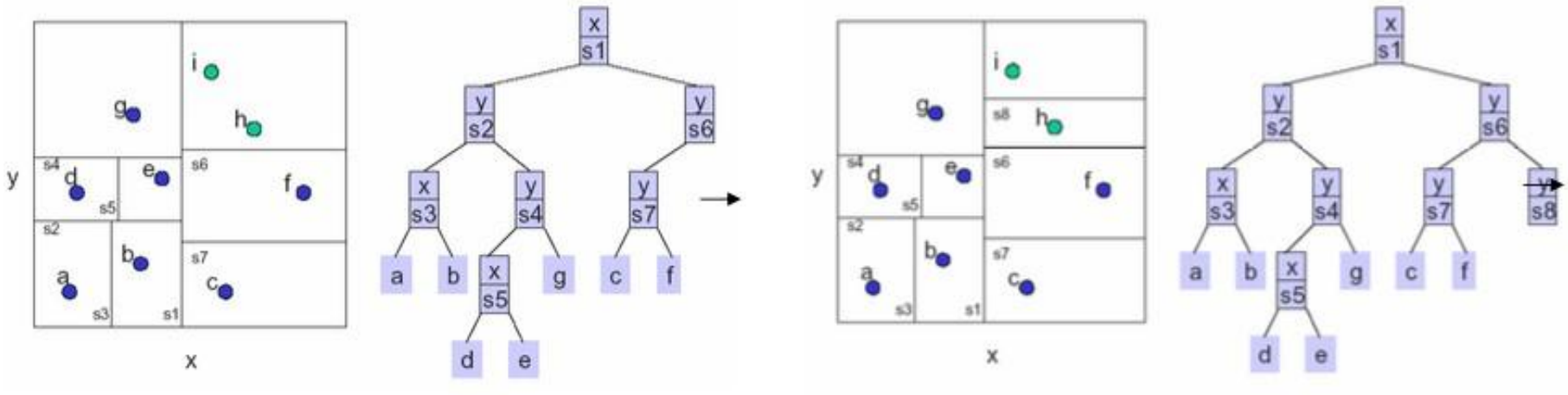
Construction



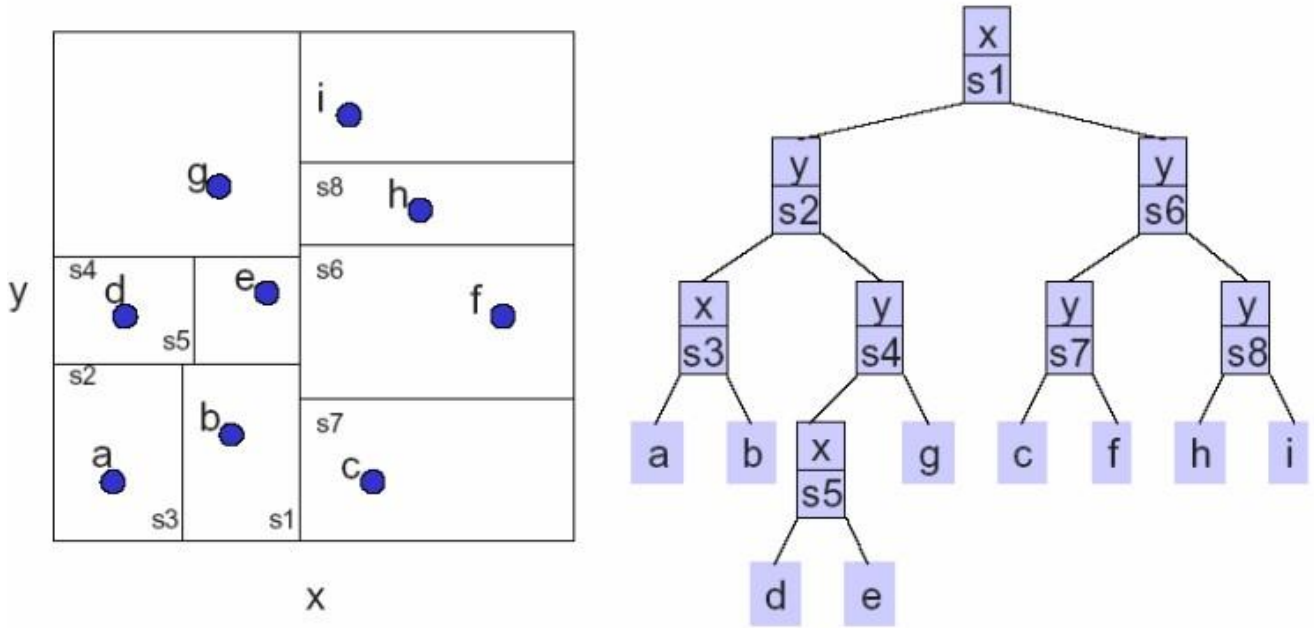
Construction



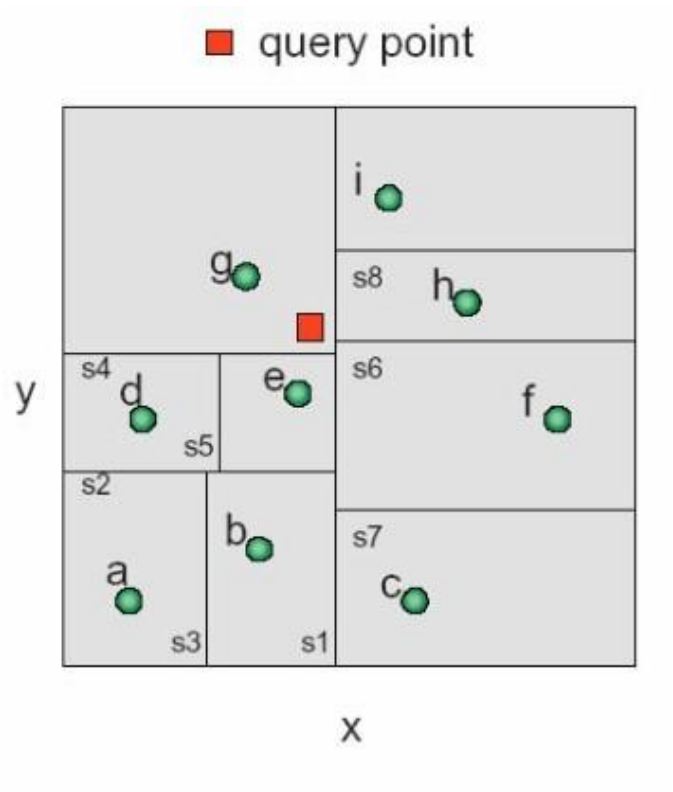
Construction

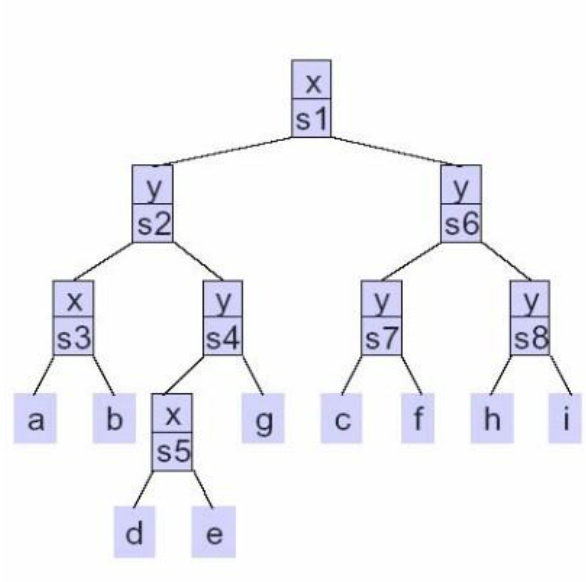


Construction

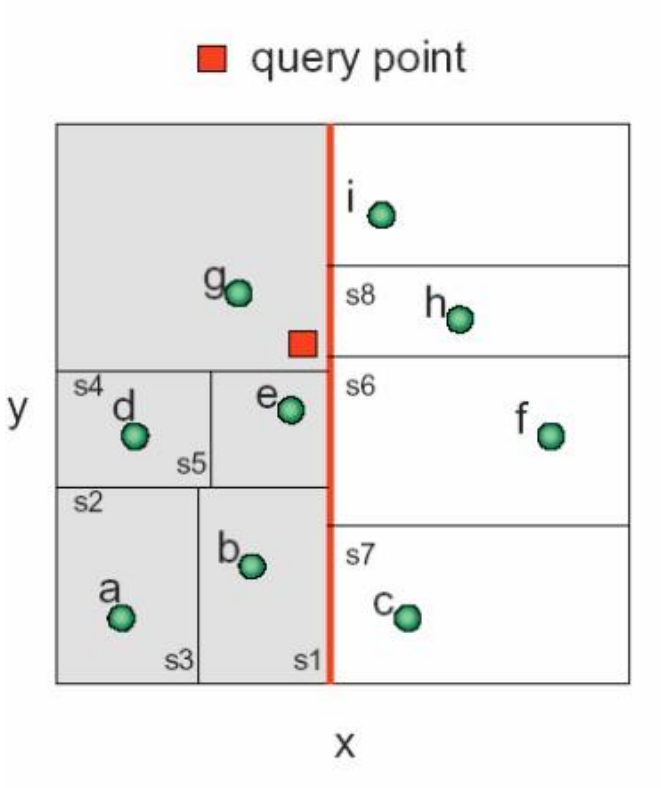


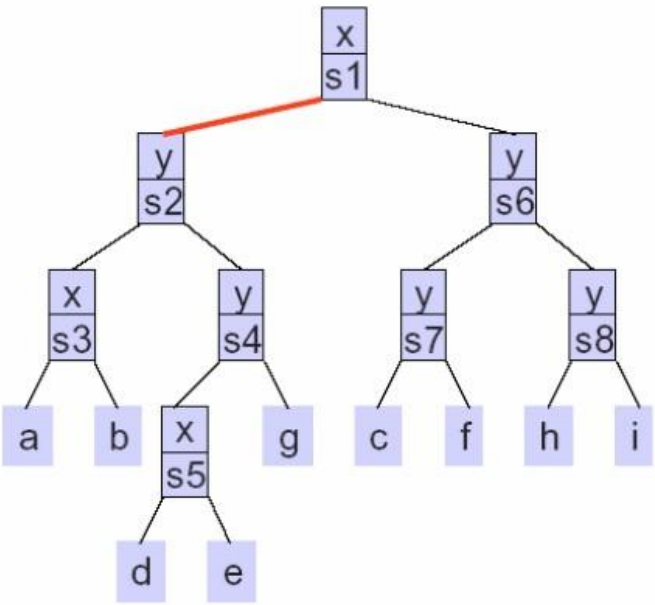
Nearest neighbor search



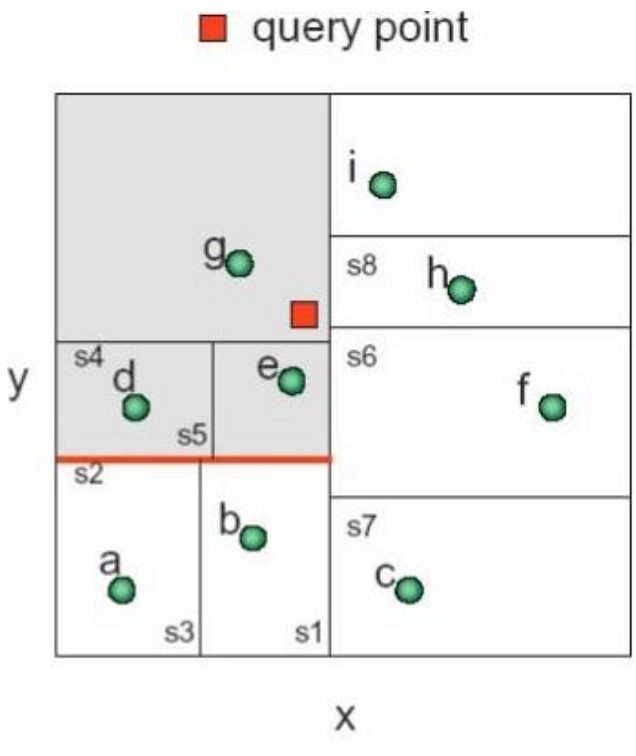


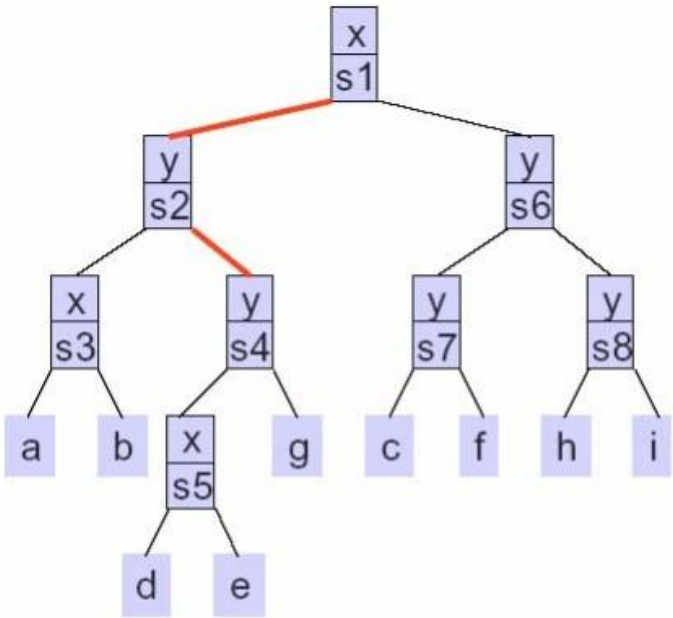
Nearest neighbor search



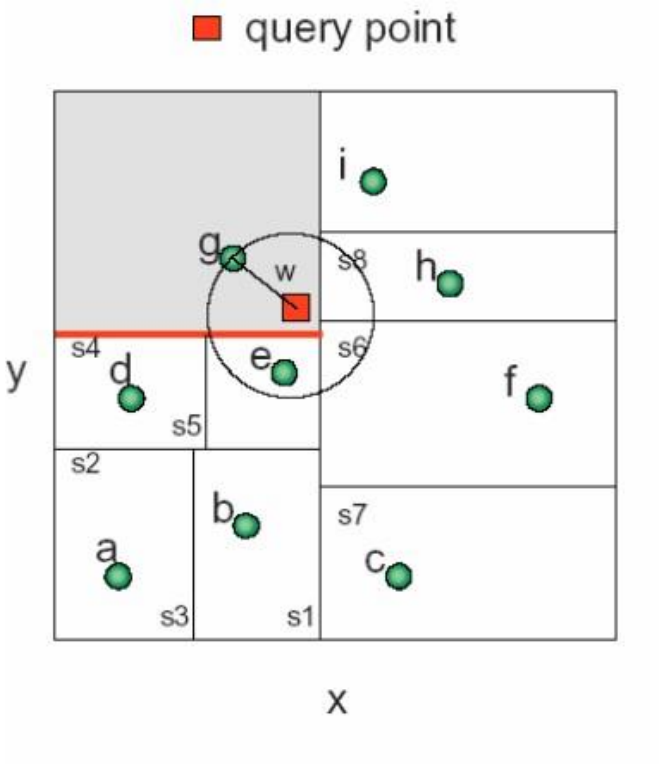


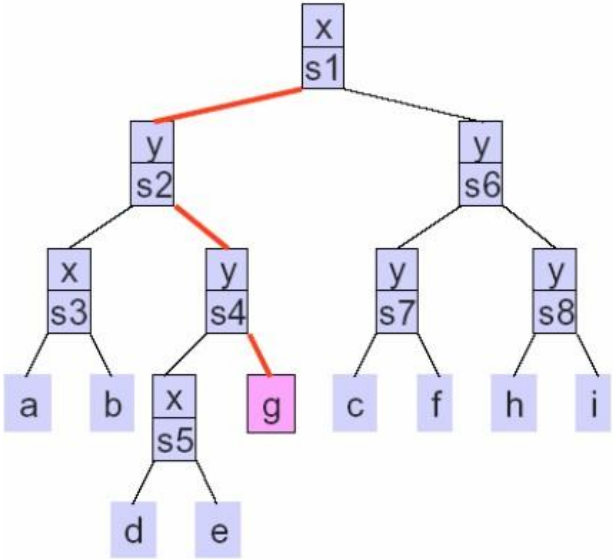
Nearest neighbor search





Nearest neighbor search

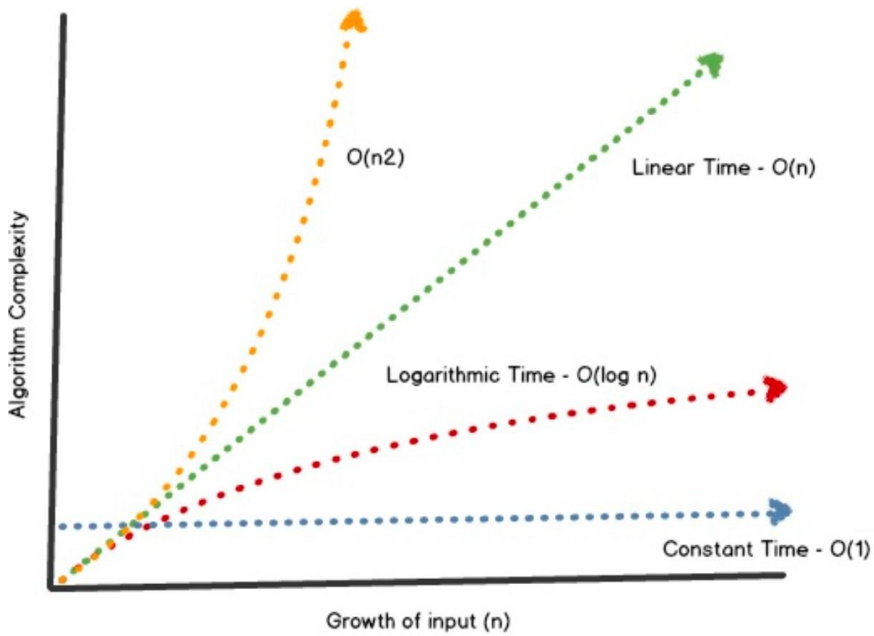




Curse of Dimensionality

- Many algorithms that work fine in low dimensions become intractable when the input is high-dimensional.
- Generalizing correctly becomes exponentially harder as the dimensionality (number of features) of the examples grows, because **a fixed-sized training set covers a dwindling fraction of the input space**.
- One might think that gathering more features never hurts, since at worst they provide no new information about the class. But in fact their benefits may be outweighed by the curse of dimensionality.

Complexity


O(n2)
Linear Time - O(n)
Logarithmic Time - O(log n)
Constant Time - O(1)
Algorithm Complexity
Growth of input (n)

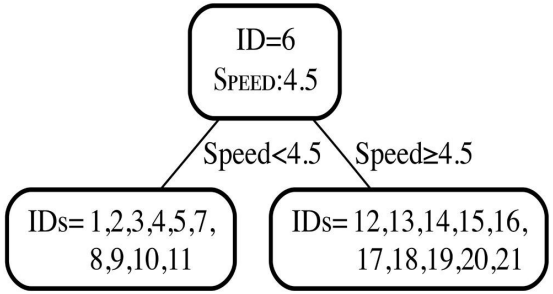
Data Structure for Improving Runtime Performance

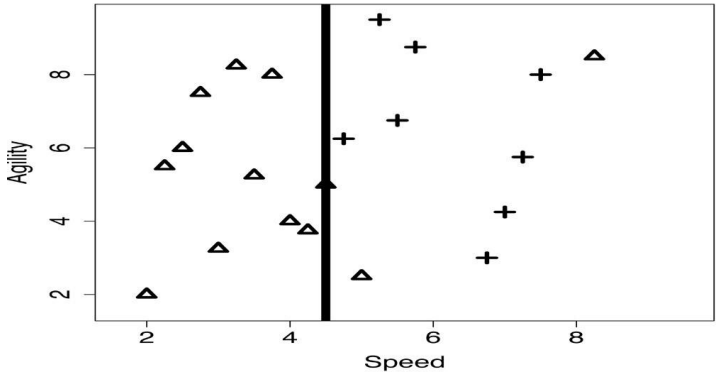
- The fact that NN algorithms store the entire training set in memory has a very negative impact on the time complexity of these algorithms.
- Structures and methods have been developed can be used to help alleviate this issue.
- One such structure is called a k-d tree, which is a balanced binary tree.
- We will use the following example to help illustrate the ideas of a k-d tree.

ID	SPEED	AGILITY	DRAFT	ID	SPEED	AGILITY	DRAFT
1	2.50	6.00	no	12	5.00	2.50	no
2	3.75	8.00	no	13	8.25	8.50	no
3	2.25	5.50	no	14	5.75	8.75	yes
4	3.25	8.25	no	15	4.75	6.25	yes
5	2.75	7.50	no	16	5.50	6.75	yes
6	4.50	5.00	no	17	5.25	9.50	yes
7	3.50	5.25	no	18	7.00	4.25	yes
8	3.00	3.25	no	19	7.50	8.00	yes
9	4.00	4.00	no	20	7.25	5.75	yes
10	4.25	3.75	no	21	6.75	3.00	yes
11	2.00	2.00	no				

Data Structure for Improving Runtime Performance

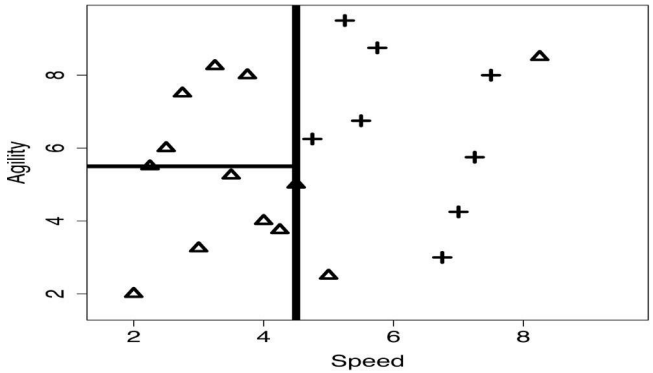
- To construct a k-d tree we put arbitrarily put all features into a list.
- We pick the first feature and split the data using the median value of that feature.
- The example below shows a simple dataset that contains a speed and agility feature.
- We initially split by speed using the median value from the data set.

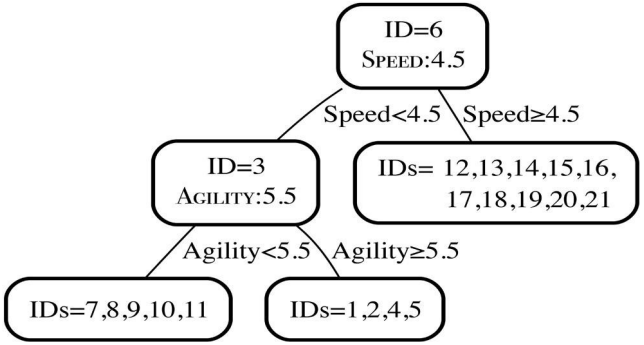


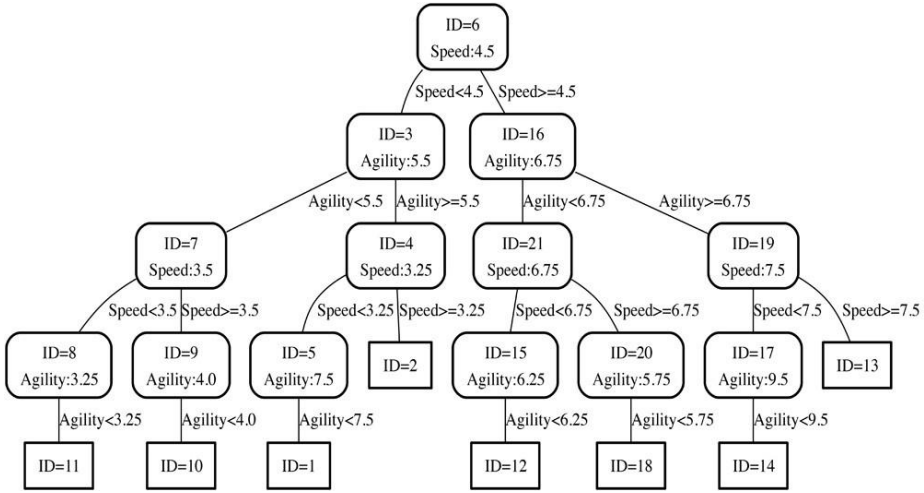


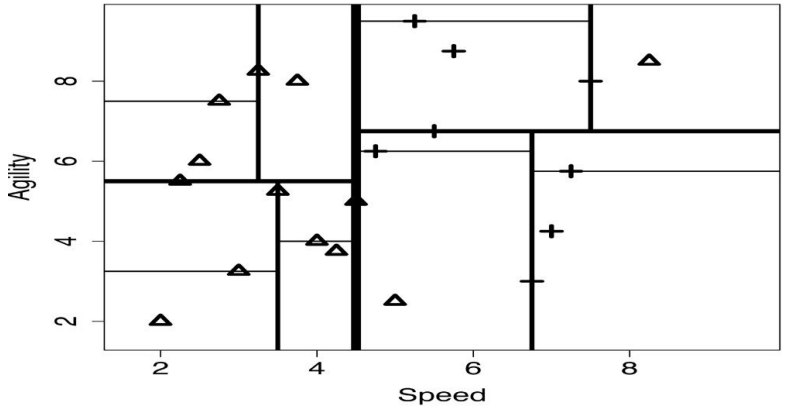
Data Structure for Improving Runtime Performance

- Every time we partition the data, we add a node with two branches to the tree. The node represents the instance that is the median value of the feature.
- The left branch holds all instances that have a value less then the median of the feature and on the right branch all instances greater than the median.

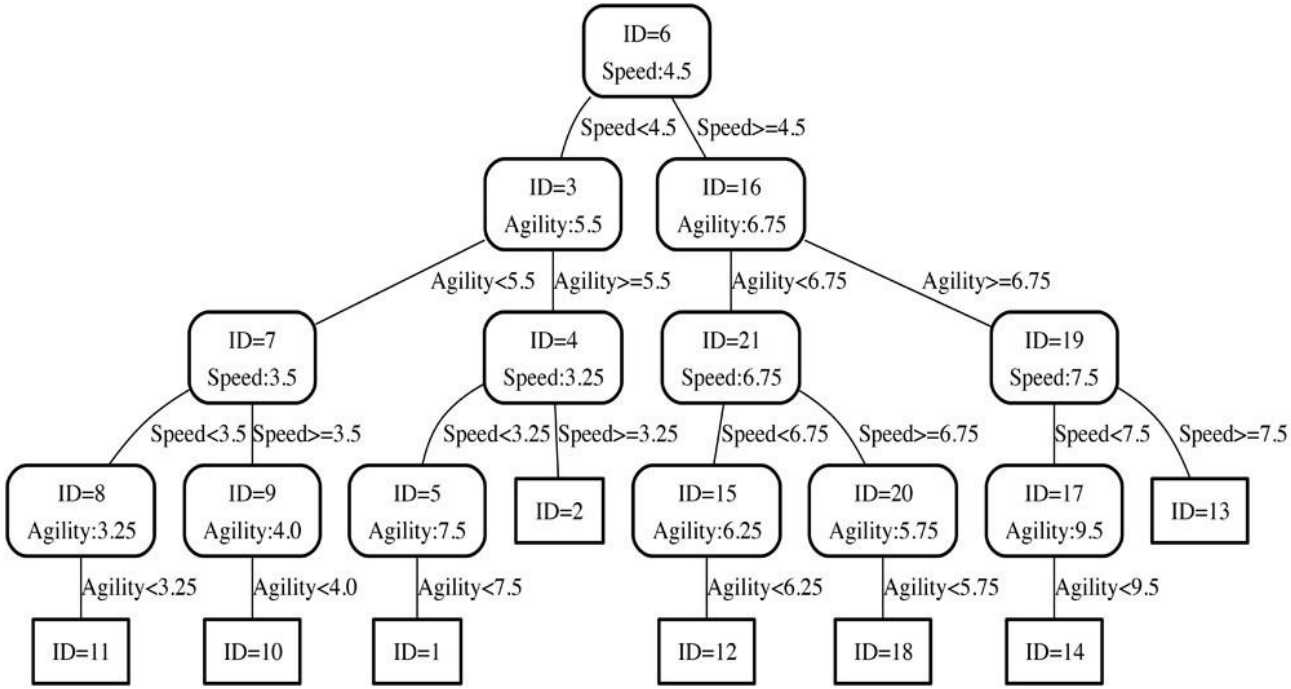




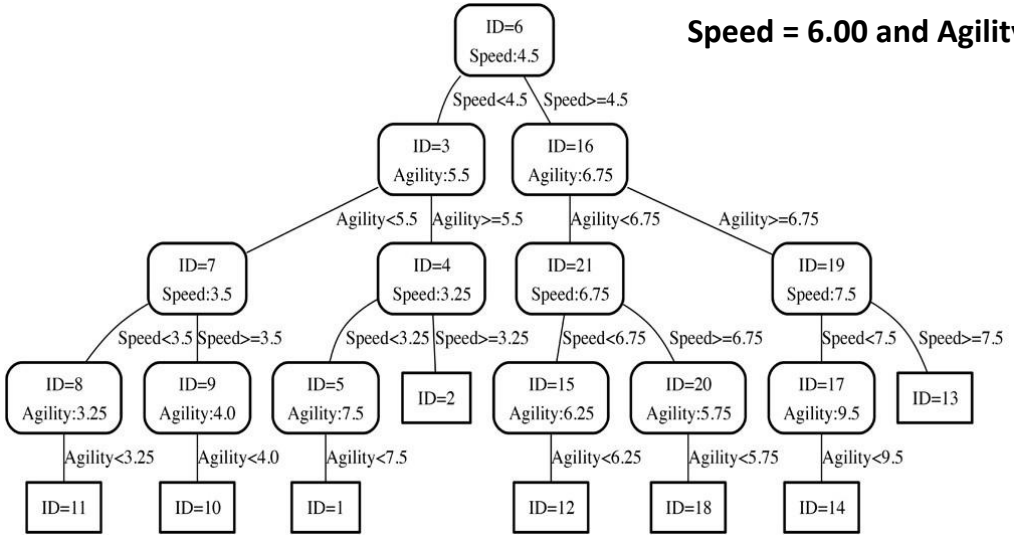

ID=6
Speed:4.5
Speed<4.5
Speed>=4.5
ID=3
Agility:5.5
ID=16
Agility:6.75
Agility<5.5
Agility>=5.5
Agility<6.75
Agility>=6.75
ID=7
Speed:3.5
ID=4
Speed:3.25
ID=21
Speed:6.75
ID=19
Speed:7.5
Speed<3.5
Speed>=3.5
Speed<3.25
Speed>=3.25
Speed<6.75
Speed>=6.75
Speed<7.5
Speed>=7.5
ID=8
Agility:3.25
ID=9
Agility:4.0
ID=5
Agility:7.5
ID=2
ID=15
Agility:6.25
ID=20
Agility:5.75
ID=17
Agility:9.5
ID=13
Agility<3.25
Agility<4.0
Agility<7.5
Agility<6.25
Agility<5.75
Agility<9.5
ID=11
ID=10
ID=1
ID=12
ID=18
ID=14

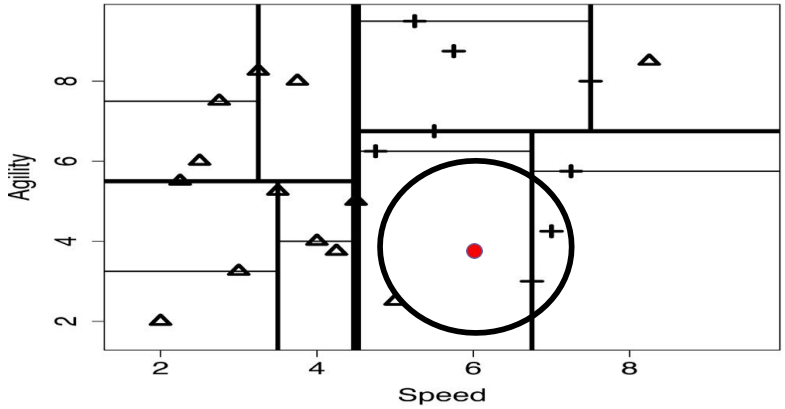

Agility
2
4
6
8
Speed
2
4
6
8

- Once we have stored the instances in a dataset in the tree we can use the tree to quickly retrieve the nearest neighbour for a query instance.
- The first step of this process is to take the query instance and follow it's path through the tree (take a query instance where Speed = 6.00 and Agility = 3.50)



Speed = 6.00 and Agility = 3.50





That's all Folks!
SUPERB wallpapers